
KNOWLEDGE BASED AGENTS: PROPOSITIONAL LOGIC

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

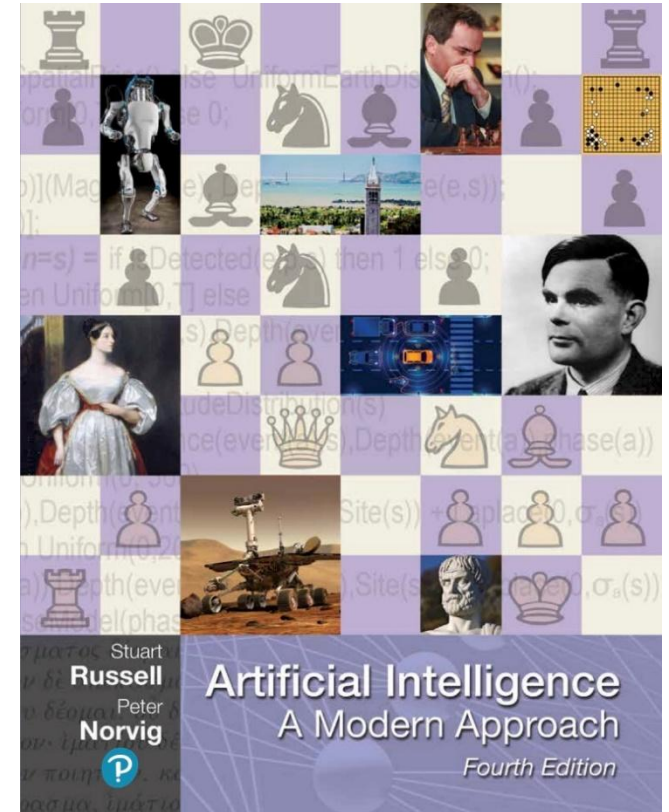
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy
- Some content adapted from Dr. Richard Lathrop (UC Irvine)

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 7 in AIMI 4/E



Outline

- Introduction to Knowledge Based Agents (KBA)
- Wumpus Game Example
- Propositional Logic: Inference, Equivalence, Validity, Satisfiability
- Inference in Propositional Logic
 - Inference by enumeration (model-checking)
 - Inference by searching using inference rules
 - Inference by Resolution-based refutation
 - Inference by Forward/Backward Chaining of horn clauses
- Limitations of Propositional Logic
- SAT Solvers
- Summary

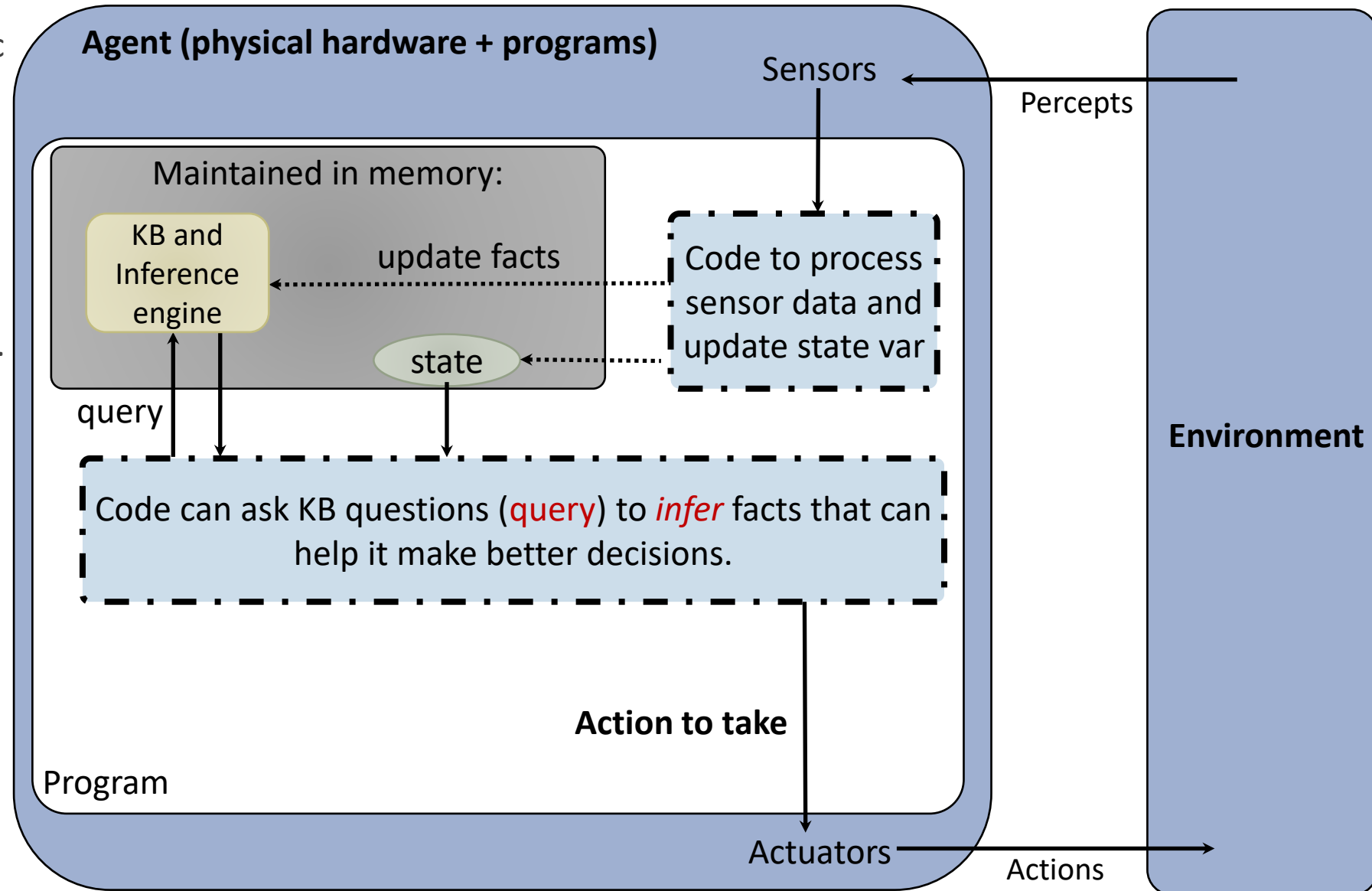
Knowledge Based (KB) Agents

What is a Knowledge-based agent?

- Knowledge based agents (KBA) use **reasoning** over an internal representation of knowledge to infer new facts about the environment.
- Problem solving agents that we've seen are limited in their reasoning.
 - Cannot **reason and infer** new facts.
 - Limited when dealing with partially observable environment. Would need to list all possible outcomes (for those interested, see Ch 4.3-4.5 in textbook)
- KBA applies logical inference (reasoning)
 - Use a set of **logical statements** to derive new ones.
 - This is done using logical **inference rules**.

KB- Agent

- KB agent maintains domain-specific facts.
- Agent can update KB facts:
 - Initial facts about the environment.
 - Facts received from percept.
 - New facts about the environment.
- Agent can query the KB:
 - Using the KB, is some statement true?
 - Using the KB, has the goal been reached?
- Querying the KB uses underlying **inference algorithm** to derive new facts from old facts. This is the main focus of this lecture.



Example: Wumpus World

Wumpus

Monster lurks somewhere in the cave, eats agent if it enters its room.

Agent(single player)

Always starts in [1,1] facing right

Goal: Explore cave to find gold, return to [1,1] and exit cave

Actions: move forward, turn left, turn right, shoot (only one arrow), grab-gold, climb (only in [1,1])

Sensors: five sensors that perceive from current agent pos

- *Stench* (occurs next to Wumpus)
- *Breeze* (occurs next to pit)
- *Glitter* (occurs in square with gold)
- *Bump* (occurs when agent moves into a wall)
- *Scream* (occurs from Wumpus when killed)

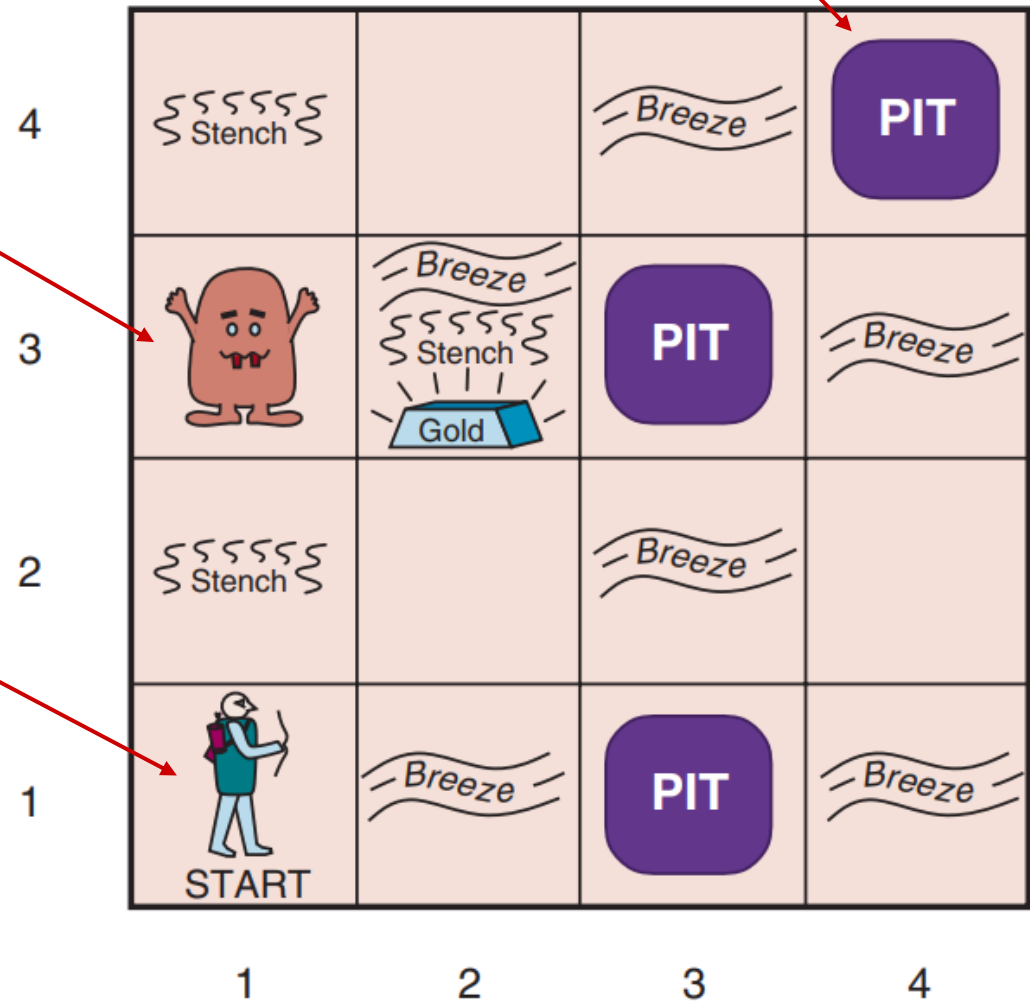
Percepts is vector [Stench, Breeze, Glitter, Bump, Scream]

Performance measure:

- +1000 for getting the gold and reaching exit,
- 1000 for being dead,
- 1 for each action taken,
- 10 for using up the arrow

PIT

Bottomless pit traps any agent that wanders into its room



Cave with NxN grid of rooms

Example: Wumpus World

- Agent has initial knowledge about the configuration:
 - Agent knows the starting location at [1,1] facing right.
 - Agent does not know where the pits are.
 - Agent does not know where Wumpus is.
 - Agent knows the rules of the environment.
- Partially-observable: some aspects of state not perceivable like agent's location, health of Wumpus, availability of arrow.
- Goal is to explore environment, make inference (reasoning) to try to find the gold.
- Random instantiations of this problem used as toy-problem in this domain.

Wumpus World – Exploring a game (informal logic language)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

(a)

- Agent's KB initially contains the rules of the environment:
 - Agent knows it is in [1,1] and [1,1] is safe. Denoted [1,1] as "A" for agent location and "OK" for safe.
- First percept: [False, False, False, False, False]; Recall percept is vector [Stench, Breeze, Glitter, Bump, Scream]
- Can **infer** that [1,2] and [2,1] are safe.
- Action: move to safe cell e.g. [2,1].

Wumpus World – Exploring a game (informal logic language)

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(b)

- In [2,1], agent perceives “Breeze”.
- Can **infer** that pit in [3,1] OR [2,2].
- Cautious agent would decide to try [1,2] because it is guaranteed safe.

Wumpus World – Exploring a game (informal logic language)

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

(a)

- In [1,2], agent perceives “Stench”.
- Is Wumpus in [1,1]? Nope, [1,1] is agent start from rules of game.
- Is Wumpus in [2,2]? Nope, stench would have been detected in [2,1]
- Can **infer** that Wumpus [1,3].
- No Breeze in [1,2]. That means, [2,2] cannot contain a pit. **Infer** that pit in [3,1].
- Safely move to [2,2]. Detects nothing. Let’s say agent then moves to [2,3]

Wumpus World – Exploring a game (informal logic language)

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(b)

- In [2,3], agent perceives Stench, Breeze, Glitter.
- Can **infer** that pit in [2,4] OR [3, 3].
- Can act with Grab gold and return to [1,1] to Climb out of cave.

What the example tells us

- KB represents knowledge about environment with set of **facts** and **rules**.
- Combine evidence (KB facts) with rules to **infer** new facts
- **Inferences** are guaranteed to be correct if
 - Evidence (KB facts) is correct
 - Rules are correct (how environment works)
 - Inference procedure is correct (logical reasoning rules)
- The example we've shown used English language. Now we'll formalize it using one type of formal logic language: propositional logic.

Logic

- **Formal language** to represent knowledge s.t. conclusions can be drawn.
- A KB is a set of **sentences**.
- **Syntax** defines legal sentences in a language/grammar.
 - E.g. in arithmetic $x+2 \geq y$ is well-formed sentence, $+x2y$ is not well-formed sentence
- **Semantics** define meaning/interpretation of each sentence.
 - i.e., define the **truth** of each sentence with respect to each **possible world**.
 - E.g.
 - $x+2 = y$ is true in a world where $x=7$ and $y=9$
 - $x+2 = y$ is false in a world where $x=7$ and $y=1$
- More examples from the language/grammar of arithmetic
 - $x+2 \geq y$ is a sentence; $x2+y > \{\}$ is not a sentence
 - $x+2 \geq y$ is true iff the number $x+2$ is no less than the number y
 - $x+2 \geq y$ is true in a world where $x = 7, y = 1$
 - $x+2 \geq y$ is false in a world where $x = 0, y = 6$

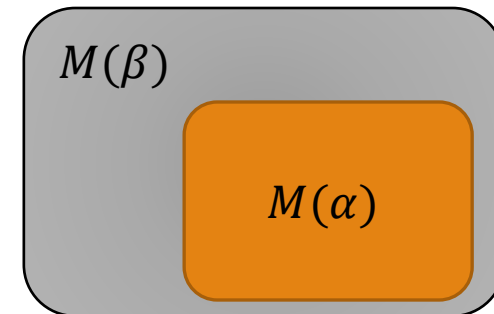
Models and Entailment

- Logicians typically think in terms of **models**.
- A model is a **configuration** that establishes the truth or falsity of a sentence.
- For example, consider the sentence $x + y = 4$. For which configurations of x and y is this true?
 - True for model $\{x = 2, y = 2\}$. Also, model $\{x = -1, y = 5\}$. Many other models are possible.
 - It is true in all models that set x and y as numbers that satisfy $x + y = 4$. False in other models.
- If a sentence α is true in a model m , then we say that m satisfies α or m is a model of α .
 - For example, the model $\{x = 2, y = 2\}$ satisfies $x + y = 4$.
- $M(\alpha)$ is the set of all models of α . That is, the set of all models in which α is true.
- To make progress towards logical reasoning (inference), we introduce the concept of **entailment**.
- **Entailment** denotes that a sentence *follows logically* from another sentence. Written as:

$$\alpha \models \beta \text{ (i.e. } \alpha \text{ “entails” } \beta \text{)}$$

- When we ask does $\alpha \models \beta$? We are asking: if we know that α is true, does it mean that β is true?
- Formally, $\alpha \models \beta$ if and only if in every model in which α is true, β is also true. That is:

$$\alpha \models \beta \text{ if and only if } M(\alpha) \subseteq M(\beta)$$



Entailment with a KB

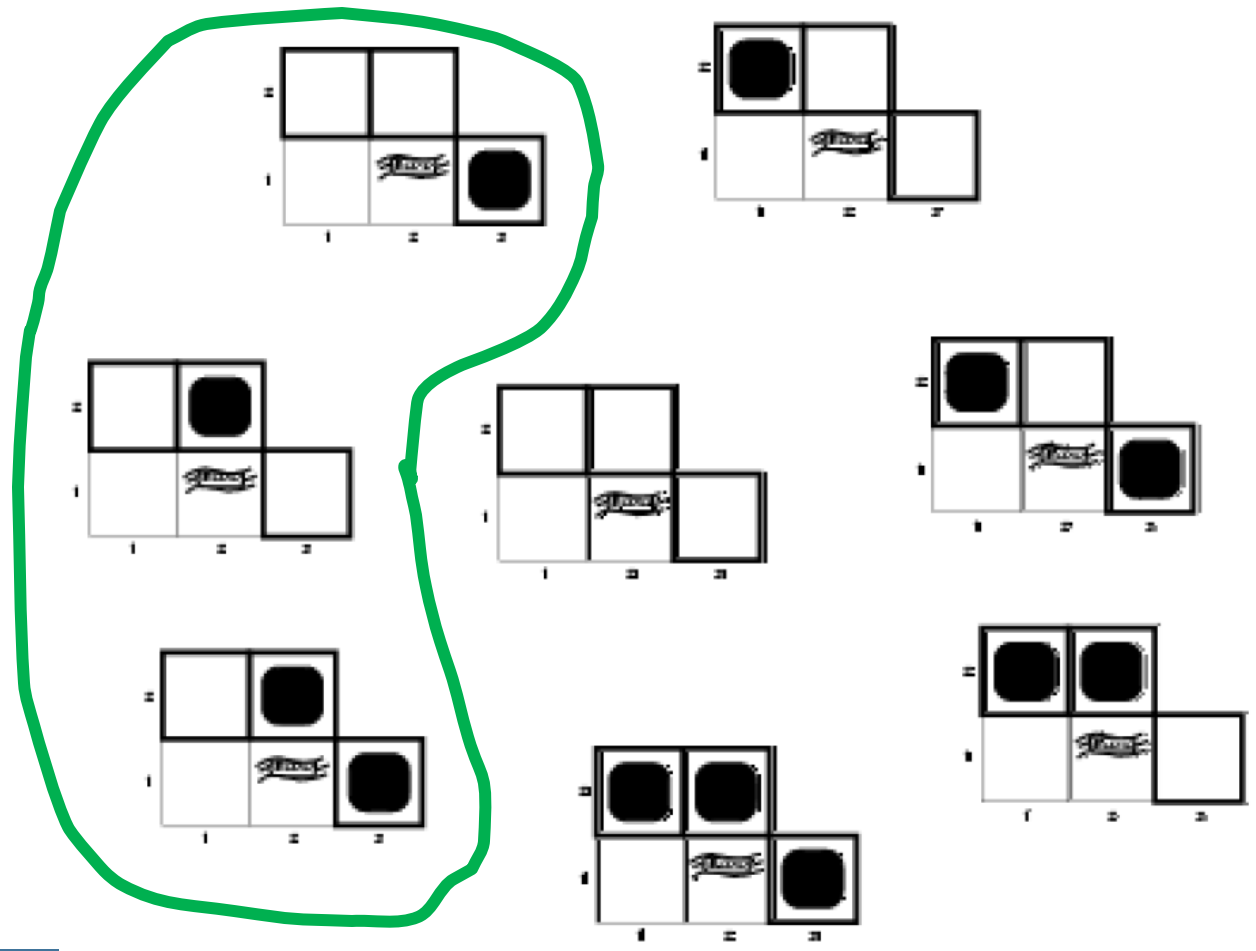
- Recall the Wumpus world example. Consider scenario on the right.
- KB contains only the following facts:
 - [2,1] has agent, breeze, and is safe.
 - Agent knows that if he detects Breeze, then there is pit in adjacent square.
 - [1,1] is safe.
 - [1,2] is safe.
 - [2,2] OR [3,1] contain pit.
- Consider the **query** “Does [1,2] or [2,2] or [3,1] contain a pit?”
- Ignoring the KB facts, how many possible models (configurations) are there for pits in these three locations?
 - 2 choices per location (pit or no-pit), thus, $2^3 = 8$ models.
- Based on KB, how many possible models are there for pits in these three locations?
 - From fact 4 in KB, agent knows there is no pit in [1,2].
 - From fact 5 in KB, pit is in both squares or one of them, so 3 possible models.
- Thus, for the given query, KB is **true** in just 3 models out of the 8 total models.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

Entailment with a KB

- From 4, agent knows there is no pit in [1,2]. From 5, pit is in both squares or one of them, so 3 possible models.
- Thus , for the given query, KB is **true** in just 3 models out of the 8 total models.

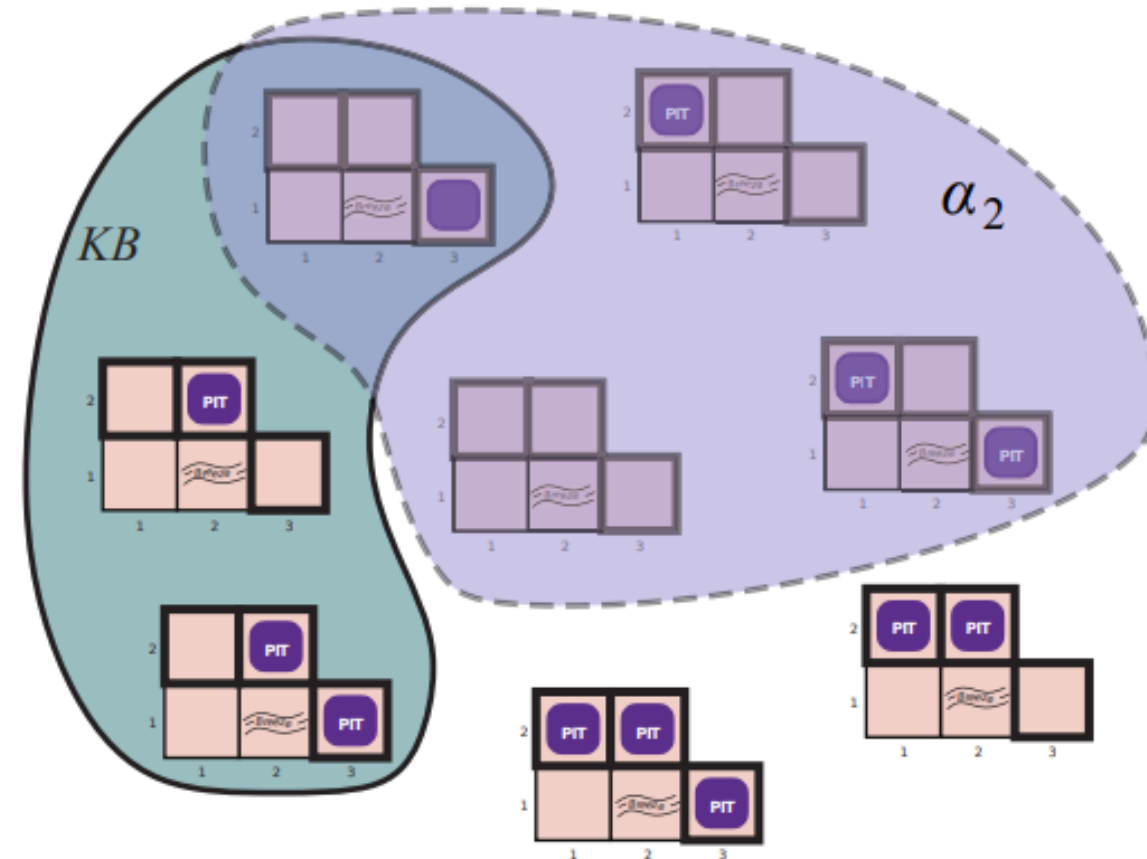
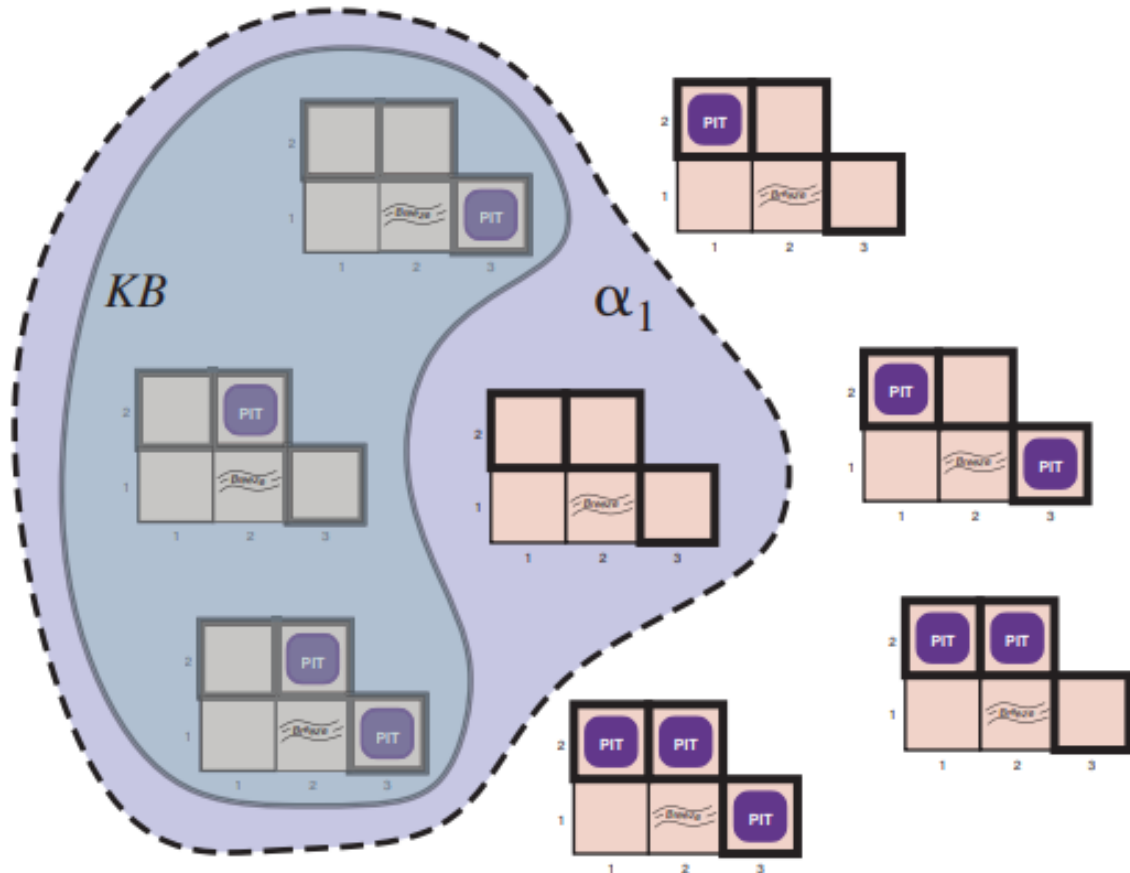
KB



1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

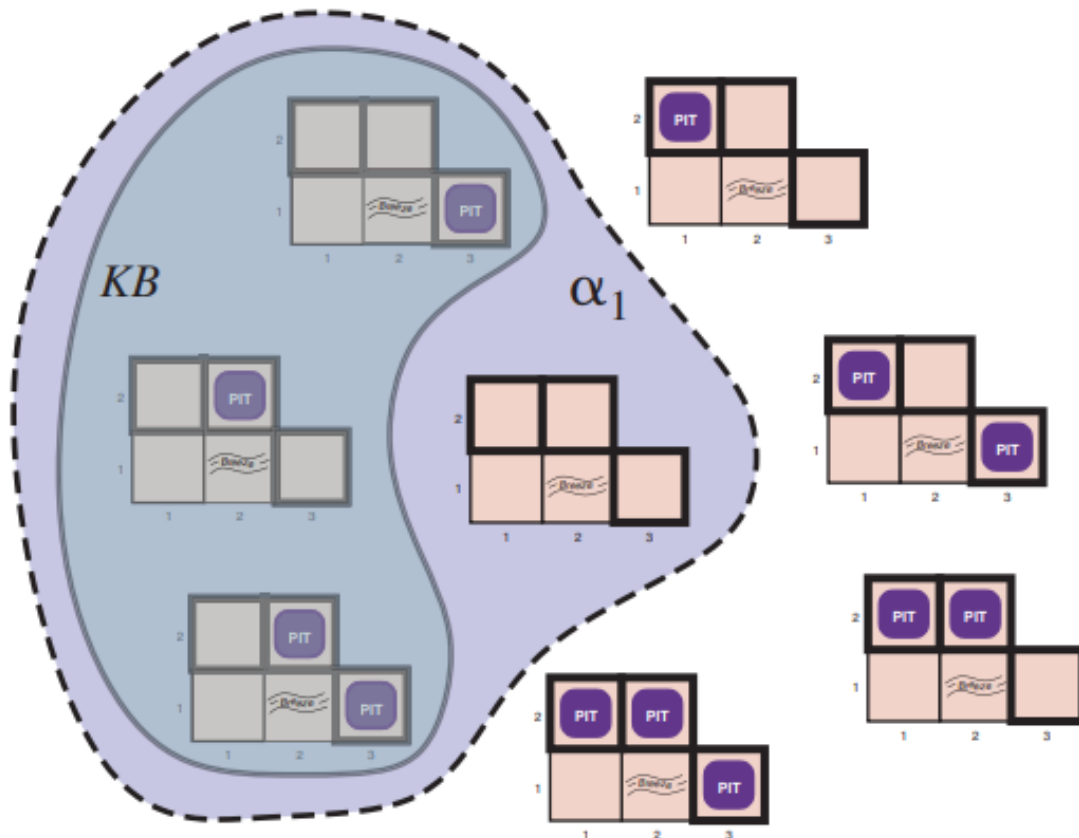
Entailment with a KB

- Now consider, two queries: α_1 = “There is no pit in [1,2]” and α_2 = “There is no pit in [2,2]”
- Out of the 8 models, α_1 is true in 4 of them.
- Out of the 8 models, α_2 is true in 4 of them.



Entailment with a KB

- We notice from the figures, that the models in which KB is true, α_1 is also true.
- Recall entailment definition. $KB \models \alpha_1$ if and only if in every model in which KB is true, α_1 is also true.
- Thus, $KB \models \alpha_1$. That is, we can **infer** from the KB that the query α_1 holds (i.e. is true).

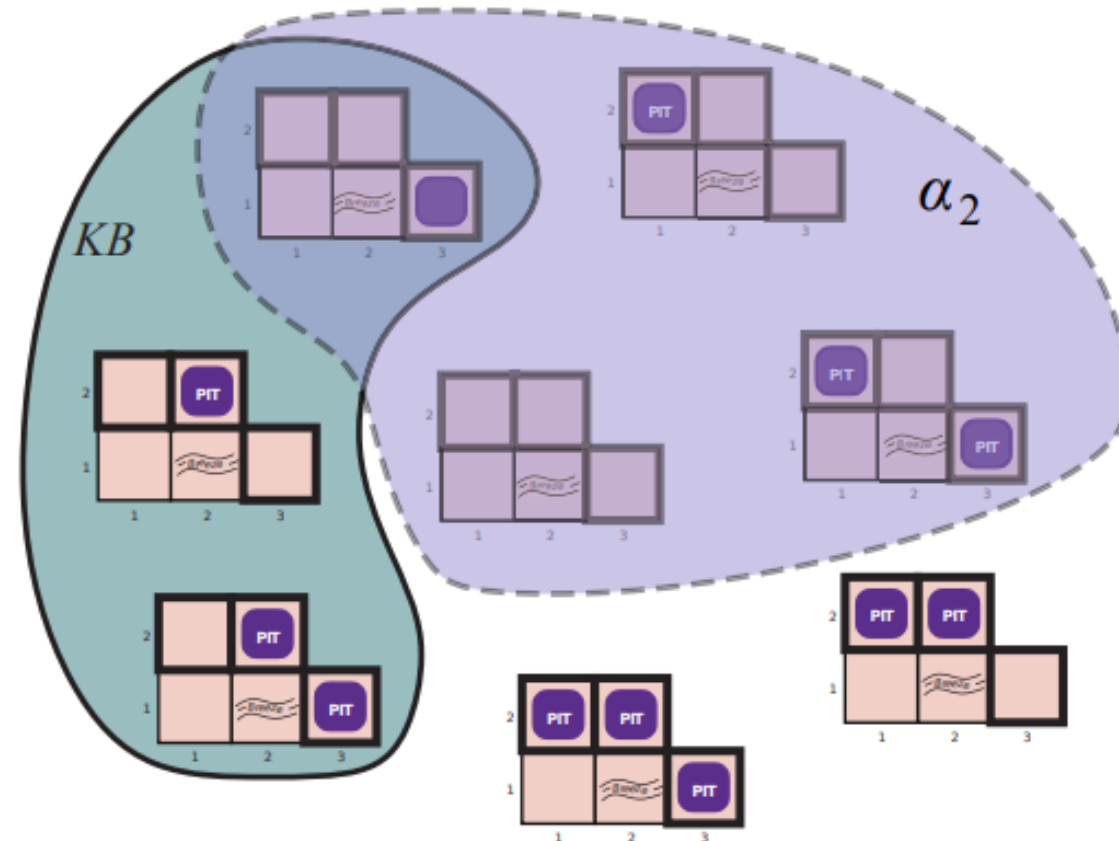


$KB \models \alpha$ if and only if $M(KB) \subseteq M(\alpha)$

Entailment with a KB

- Recall entailment definition. $KB \models \alpha$ if and only if in every model in which KB is true, α is also true.
- Does $KB \models \alpha_2$?
- No! there are 3 models that do not overlap!
- Thus, we say that $KB \not\models \alpha_2$. That is, KB does not entail α_2 .
- This does **not** say that α_2 is false!
- All this says is that the KB cannot infer the truth of α_2 . In other words, the KB does not know the status of α_2 .
- Perhaps as the agent moves in the environment and adds more evidence in the KB, then it will shrink its set of possible models. Then perhaps α_2 can be determined.

$KB \models \alpha$ if and only if $M(KB) \subseteq M(\alpha)$



Entailment with a KB

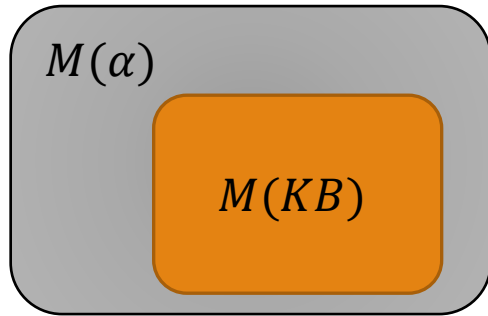
- This example shows how we can use entailment to derive conclusions. That is, to establish the truth of a query using the facts in the KB.
- This process is called **logical inference**.
- This specific process (algorithm) we just showed of enumerating all possible models is called **model-checking**.
- Because it enumerates all possible models to check that α is true in all models in which KB is true.

$$KB \models \alpha \text{ if and only if } M(KB) \subseteq M(\alpha)$$

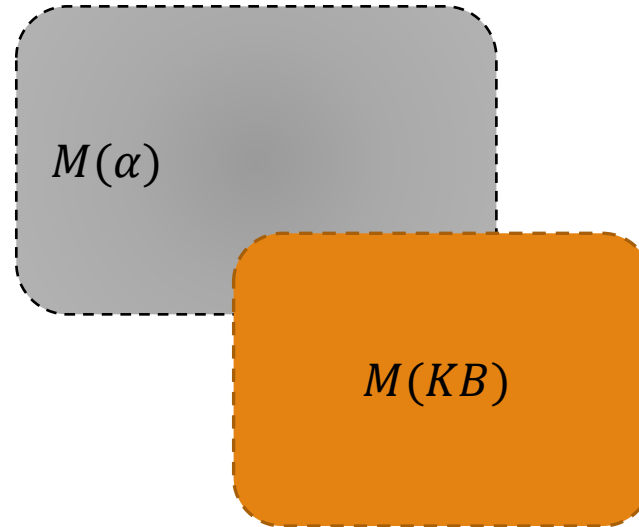
Does $KB \models \alpha$?

$KB \models \alpha$ if and only if $M(KB) \subseteq M(\alpha)$

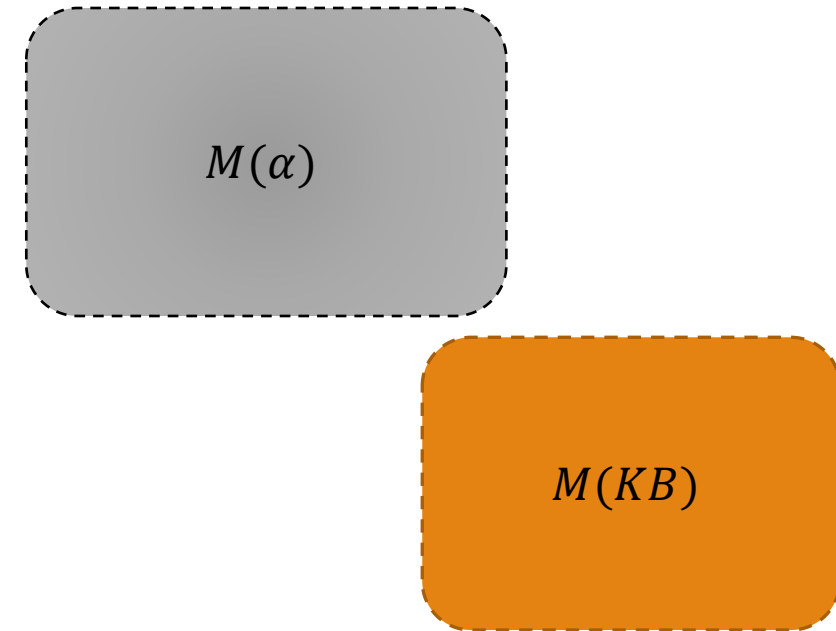
Three scenarios



Yes, since $M(KB) \subseteq M(\alpha)$
ALL models in which KB is
true, α is true.



No, since $M(KB) \not\subseteq M(\alpha)$.
Some model in which KB is true,
 α is false.



No, since $M(KB) \not\subseteq M(\alpha)$.
ALL models in which KB is true,
 α is false.

Entailment with a KB – Using inference algorithms

- Algorithms for establishing entailment are called **inference algorithms**.
- An inference algorithm i that **derives** α from KB is denoted as $KB \vdash_i \alpha$.
- Two desirable properties of inference algorithms: **soundness** and **completeness**.
- An inference algorithm is **sound** if it **derives** only entailed sentences.
 - That is, it does not derive not-entailed sentences. Otherwise, we cannot use it to establish entailment. An unsound algorithm can derive whatever it wants regardless of entailment!
- An inference algorithm is **complete** if it can **derive** any sentence that is entailed.
- Thus, an inference algorithm can be sound but not complete. That is, if it does derive a sentence, then entailment holds. But there might be entailed sentences that it cannot derive.
- An inference algorithm can be complete but not sound.
- Desirable for algorithm to be both.
 - Refusing to infer any sentence is Sound; so, Sound is weak alone.
 - Deriving every sentence is Complete; so, Complete is weak alone.

Soundness: i is sound if

whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$

Completeness: i is complete if

whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$

Overview of what we'll cover

- We want to cover inference algorithms that can be used to infer new facts from a KB.
- This requires the use of a logic language, and then appropriate inference algorithms for that language.
- Two logic languages: propositional logic and first order logic.
- Inference algorithms divide into *roughly* two kinds:
 1. Truth table enumeration (called **model checking**). Exponential time complexity.
 2. Theorem proving algorithms: apply rules of inference directly to sentences in KB to construct a proof of the desired query sentence.
 - By deduction (direct proof). Worst-case exponential.
 - By contradiction. Known as resolution-based prover. Worst-case exponential. Can sometimes be fast with modern SAT solver.
 - Forward/Backward chaining. Restricted to KB with horn clauses. Linear time algorithms. Used by logic programming systems like [Prolog](#).

Inference in Propositional Logic

Propositional Logic – Syntax

- Propositional logic is a logic language that can be used to represent sentences (syntax and semantics).
- A proposition is a statement (sentence) that is either true or false.
- Syntax of PL defines the allowable grammar of sentences.
- **Atomic** sentences are single proposition symbols
 - E.g., P, Q, R, FacingEast. Convention is to use uppercase letters.
 - Special symbols: *true* = always-true proposition, *false* = always-false proposition
- Complex sentences are formed using other sentences and logical operators/connectives:

$\neg S$	negation ; not operation
$S_1 \wedge S_2$	conjunction ; and operation; S1, S2 are called conjuncts
$S_1 \vee S_2$	disjunction ; or operation; S1, S2 are called disjuncts
$S_1 \Rightarrow S_2$	implication ; rule, if-then statement, conditional; S1 is called premise or antecedent, S2 is called conclusion or consequence.
$S_1 \Leftrightarrow S_2$	Bi-conditional; if and only if

Propositional Logic – Syntax

- BNF (Backus-Naur Form) grammar defines the syntax of PL.

$$\textit{Sentence} \rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence}$$
$$\textit{AtomicSentence} \rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \dots$$
$$\textit{ComplexSentence} \rightarrow (\textit{Sentence})$$
$$\mid \neg \textit{Sentence}$$
$$\mid \textit{Sentence} \wedge \textit{Sentence}$$
$$\mid \textit{Sentence} \vee \textit{Sentence}$$
$$\mid \textit{Sentence} \Rightarrow \textit{Sentence}$$
$$\mid \textit{Sentence} \Leftrightarrow \textit{Sentence}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

- Note operator precedence. Can use parentheses to resolve ambiguity.

$\neg P \vee Q \wedge R \Rightarrow S$ is equivalent to $((\neg P) \vee (Q \wedge R)) \Rightarrow S$

Propositional Logic – Semantics

- PL semantics define the truth-value (true or false) of a sentence.
- In PL, a **model** is a true/false **configuration** of the symbols.
- For example, if we have symbols P, Q, R. Then one possible model is $m_1 = \{P = \text{true}, Q = \text{false}, R = \text{true}\}$
- Example for two propositions P and Q:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

Rules for evaluating truth with respect to a model m :

$\neg S$ is true iff	S is false
$S_1 \wedge S_2$ is true iff	S_1 is true and S_2 is true
$S_1 \vee S_2$ is true iff	S_1 is true or S_2 is true
$S_1 \Rightarrow S_2$ is true iff	S_1 is false or S_2 is true
i.e., is false iff	S_1 is true and S_2 is false
$S_1 \Leftrightarrow S_2$ is true iff	$S_1 \Rightarrow S_2$ is true and $S_2 \Rightarrow S_1$ is true

Simple recursive process evaluates an arbitrary sentence, e.g.,

$$\begin{aligned}\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) &= \neg true \wedge (true \vee false) \\ &= false \wedge true = false\end{aligned}$$

Propositional Logic – Inference Algorithms

- We use inference algorithms to establish $KB \models \alpha$
- Inference algorithms divide into *roughly* two kinds:
 1. Truth table enumeration (called **model checking**)
 - Exponential in the number of propositions $O(2^n)$
 2. Theorem proving algorithms: apply rules of inference directly to sentences in KB to construct a proof of the desired query sentence.
 - By deduction (direct proof)
 - By contradiction
- We will discuss model checking first.

Proof by truth table enumeration (model checking) – example

- Given $KB = A \vee C, B \vee \neg C$
- Given $\alpha = A \vee B$
- Does $KB \models \alpha$?

A	B	C	$A \vee C$	$B \vee \neg C$	KB
False	False	False	False	True	False
False	False	True	True	False	False
False	True	False	False	True	False
False	True	True	True	True	True
True	False	False	True	True	True
True	False	True	True	False	False
True	True	False	True	True	True
True	True	True	True	True	True

- KB is true when ALL of its sentences are true.

Proof by truth table enumeration (model checking) – example

- Given $KB = A \vee C, B \vee \neg C$
- Given $\alpha = A \vee B$
- Does $KB \models \alpha$?

						α	
A	B	C	$A \vee C$	$B \vee \neg C$	KB	$A \vee B$	$KB \Rightarrow \alpha$
False	False	False	False	True	False	False	True
False	False	True	True	False	False	False	True
False	True	False	False	True	False	True	True
False	True	True	True	True	True	True	True
True	False	False	True	True	True	True	True
True	False	True	True	False	False	True	True
True	True	False	True	True	True	True	True
True	True	True	True	True	True	True	True

- $KB \models \alpha$ if all models in which KB is true, α is also true. i.e. if all rows where KB is true, α is also true.

Proof by truth table enumeration (model checking) – Wumpus example

- Now consider again the Wumpus example. Let us apply model checking in propositional logic.

Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

In the situation shown in the figure, we just consider the relevant proposition symbols:

$B_{1,1}, B_{2,1}, P_{1,1}, P_{1,2}, P_{2,1}, P_{2,2}, P_{3,1}$

The knowledge base has the following sentences:

- $R_1:$

$\neg P_{1,1}$

“no pit in $[1,1]$ from rules of game”

$R_2:$

$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

“A square is breezy if and only if there is an adjacent pit”

$R_3:$

$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

“A square is breezy if and only if there is an adjacent pit”

$R_4:$

$\neg B_{1,1}$

“from percepts”

$R_5:$

$B_{2,1}$

“from percepts”

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

- How many possible models; i.e. configurations of $B_{1,1}, B_{2,1}, P_{1,1}, P_{1,2}, P_{2,1}, P_{2,2}, P_{3,1}$? $2^7=128$
- Does $KB \models \neg P_{1,2}$? We can enumerate all models.

Proof by truth table enumeration (model checking) – Wumpus example

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	false	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	true	true	true	true	true	true	<u>true</u>
false	true	false	false	true	false	false	true	false	false	true	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	true	true	false	true	false

- Enumerate all models (truth-value configurations of symbols).
- Compute truth-value of sentences in KB. Compute truth-value of KB (true when all of its sentences are true).
- If KB is true in row, check that α is true as well. We can see in table that $KB \models \neg P_{1,2}$

```
function TT-ENTAILS?( $KB, \alpha$ ) returns true or false
  inputs:  $KB$ , the knowledge base, a sentence in propositional logic
            $\alpha$ , the query, a sentence in propositional logic
   $symbols \leftarrow$  a list of the proposition symbols in  $KB$  and  $\alpha$ 
  return TT-CHECK-ALL( $KB, \alpha, symbols, []$ )

function TT-CHECK-ALL( $KB, \alpha, symbols, model$ ) returns true or false
  if EMPTY?( $symbols$ ) then
    if PL-TRUE?( $KB, model$ ) then return PL-TRUE?( $\alpha, model$ )
    else return true
  else do
     $P \leftarrow$  FIRST( $symbols$ );  $rest \leftarrow$  REST( $symbols$ )
    return TT-CHECK-ALL( $KB, \alpha, rest, \text{EXTEND}(P, \text{true}, model)$ ) and
           TT-CHECK-ALL( $KB, \alpha, rest, \text{EXTEND}(P, \text{false}, model)$ )
```

- Model checking by enumeration is **sound** and **complete inference algorithm**. What's the catch?
- n symbols means 2^n models, so time-complexity is $O(2^n)$; space complexity is $O(n)$ since it is DFS type algorithm.
- Thus, we need more efficient inference algorithms. Is this possible?

*“Unfortunately, propositional entailment is co-NP-Complete (probably no easier than NP-Complete). Every known inference algorithm for entailment in propositional logic has a **worst-case** complexity that is exponential in size of input.”*

Theorem proving inference algorithms

- Apply inference rules directly to sentences in KB to construct a proof of the desired sentence.
- *In practice, can* be more efficient than model checking. Number of models is large, but proof is short. However *worst-case* remains exponential in propositional logic.
- Legitimate (**sound**) generation of new sentences from old.
- Can use **inference rules** as operators in a standard search algorithm.
- Important concepts: **logical equivalence, validity, deduction theorem, satisfiability**

Concept: Logical Equivalence

- Useful rules for converting forms of sentences.
- Two sentences are logically equivalent iff they are true in the same set of models.

$\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	implication elimination
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

Concepts: Validity and Satisfiability

- A sentence is **valid** if it is true in **all** models (**tautology**)
 - e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$
- A sentence is **satisfiable** if it is true in **some** model.
 - e.g., $A \vee B$ is true when $A = \text{true}$.
 - Determining satisfiability of a sentence is known as SAT-problem. It is NP-complete.
- A sentence is **unsatisfiable** if it is false in ALL models.
 - e.g., $A \wedge \neg A$
 - Determining unsatisfiability of a sentence is known as UNSAT-problem. It is CoNP-complete.
- Validity and satisfiability can be used to establish $KB \models \alpha$. How?
- **Validity** is connected to inference via the **Deduction Theorem**: $KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid
 - That is, we show that $(KB \Rightarrow \alpha)$ is true in ALL models. Can be done with model checking (enumeration).
- **Satisfiability** is connected to entailment via the following: $KB \models \alpha$ if and only if $(KB \wedge \neg \alpha)$ is unsatisfiable
 - That is, we show that there is no model for which KB is true and α is false.
 - This is called proof by contradiction/refutation. Assume α is false, which leads to contradiction with KB.

Concept: Inference Rules

- [Inference rules](#) allow us to take a set of sentences as input and then output a single sentence.
- They can be depicted in two ways:

$$\begin{array}{c} \text{Premise\#1} \\ \text{Premise\#2} \\ \dots \\ \text{Premise\#n} \\ \hline \text{Conclusion} \end{array} \qquad \frac{\text{Premise\#1, Premise\#2, ..., Premise\#n}}{\text{Conclusion}}$$

- The sentences before the line are called premises, and they are used to draw the conclusion sentence.
- Can also be written as (Premise#1), (Premise#2), ..., (Premise#n) $\vdash$ conclusion.
- Semantically, what it means: if the premises are ALL true according to the KB, then the conclusion can be taken as true.
- This tells us that if in our KB we have matching premises, then we can add the conclusion to the KB as a fact.
- So, what are some known inference rules we can use?

Example of Inference Rules [1]

- Modus Ponens rule:
$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

- AND-Elimination (Simplification) rule:
$$\frac{\alpha \wedge \beta}{\alpha}$$

- Modus tollens rule:
$$\frac{\alpha \implies \beta, \quad \neg \beta}{\neg \alpha}$$

- All logical equivalences can be used as inference rules in **both directions**:

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{and} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

- See table of inference rules at [link](#).

Example of Inference Rules [2]

- Unit resolution rule:

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k},$$

where each ℓ is a literal and ℓ_i and m are **complementary literals**

- Full resolution rule:

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

- Example:

$$\frac{A \vee B \vee C, \quad A \vee \neg B \vee D}{A \vee C \vee D}$$

Note we replaced $A \vee A$ with A using equivalence

- Resolution will be helpful for an inference algorithm we will discuss.
- Terms to know: a **literal** is a single letter with or without a negation. A **clause** is a disjunction of literals.
- See table of inference rules at [link](#).

- Full resolution rule:

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

Only Resolve ONE Literal Pair!

If more than one pair, result always = TRUE.

Useless!! Always simplifies to TRUE!!

No!

$(A \vee B \vee C \vee D)$
 $(\neg A \vee \neg B \vee \neg C)$

 (D)

No! This is wrong!

Yes! (but = TRUE)

$(A \vee B \vee C \vee D)$
 $(\neg A \vee \neg B \vee \neg C)$

 $(A \vee \neg A \vee B \vee \neg B \vee D)$

Yes! (but = TRUE)

Applying inference rules to Wumpus example

- Recall the Wumpus example. Let us apply inference rules to show a **direct proof** that $KB \models \neg P_{1,2}$

KB has the following sentences:

- $R_1: \neg P_{1,1}$
- $R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- $R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- $R_4: \neg B_{1,1}$
- $R_5: B_{2,1}$

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

- Using the KB facts and inference rules we can proceed as follows:

- $R_6: (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$ biconditional elimination with R_2
- $R_7: ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$ And-elimination with R_6
- $R_8: (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$ contrapositive logical equivalence with R_7
- $R_9: \neg(P_{1,2} \vee P_{2,1})$ Modus ponens with R_8 and R_4 $\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$
- $R_{10}: \neg P_{1,2} \wedge \neg P_{2,1}$ De Morgan's with R_9
- $R_{11}: \neg P_{1,2}$ And-elimination with R_{10}

Direct Proof Inference Algorithm

- From the previous example, we see that we can establish $KB \models \alpha$ by applying inference rules to sentences in the KB until α is added to the KB. This is **direct proof** inference algorithm.
- During this incremental process, the KB is being **increased** with more sentences. The application of inference rules is similar to actions.
- This means we can use search algorithms (BFS, DFS, etc.) to search for these proofs by modelling it as a search problem:
 - **INITIAL STATE**: the initial knowledge base.
 - **ACTIONS**: the set of actions consists of all the inference rules applied to all the sentences that match the top half of the inference rule.
 - **RESULT**: the result of an action is to add the sentence in the bottom half of the inference rule.
 - **GOAL**: the goal is a state that contains the sentence we are trying to prove.
- This can be more efficient in practice than enumeration because we might get to the goal without involving a large number of symbols. However, the **worst-case** can still be exponential (Why? Recall search algorithm complexity).
- Using search algorithms with inference rules is a **sound inference algorithm**. Is it **complete**? If the inference rules are sufficient to get to the goal, and we use a complete search algorithm, then yes.

Entailment using a single Inference Rule – Resolution

- Can we get away with using just one inference rule and achieve a sound and complete algorithm?
- Yes. The resolution algorithm uses the resolution inference rule, and is sound and complete.

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

- The resolution rule only applies to clauses (disjunction of literals).
- Any sentence in propositional logic can be converted to a disjunction of conjunctions. That is, into **Conjunctive-Normal-Form (CNF)**.
- So, if we convert all the sentences in the KB to CNF, then we can just apply a single inference rule (i.e. resolution).

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Resolution algorithm

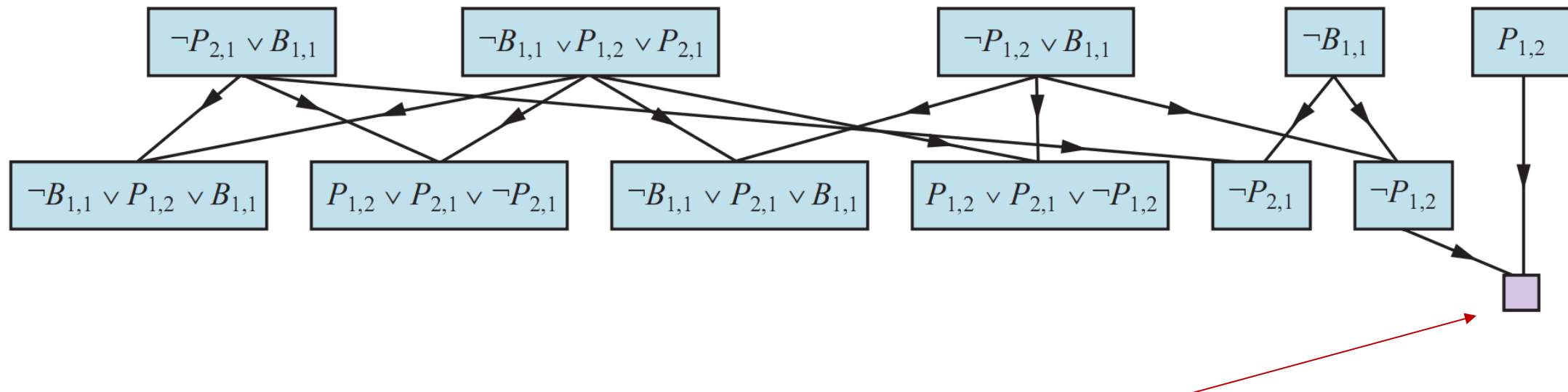
- Recall that earlier we said: $KB \models \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable
- To establish $KB \models \alpha$, the resolution algorithm uses a proof by contradiction to show that $(KB \wedge \neg\alpha)$ is **unsatisfiable**.
- Resolution algorithm works as follows:
 - First step, convert $(KB \wedge \neg\alpha)$ to CNF.
 - Then the resolution rule is applied to pairs of clauses that contain complimentary literals, and the resulting clause is added to the KB. This is done again and again until:
 - Two clauses resolve to yield the *empty* clause. This means that $(KB \wedge \neg\alpha)$ is unsatisfiable, which means that $KB \models \alpha$.
 - No new clauses can be added to the KB. This means that $KB \not\models \alpha$.
- A resolution-based inference algorithm is **sound and complete** for propositional logic.
 - Sound because resolution rule is sound.
 - Complete due to the **ground resolution theorem**. This means that algorithm terminates. For those interested, proof in textbook pg. 246.

Resolution algorithm – Example

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \quad \alpha = \neg P_{1,2}$$

KB in CNF $((\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1}) \wedge \neg B_{1,1})$

Illustration of resolution algorithm being applied to $KB \wedge \neg\alpha$ repeatedly until empty clause is generated.



Empty clause is produced, thus $KB \models \neg P_{1,2}$

```
function PL-RESOLUTION( $KB, \alpha$ ) returns true or false  
  inputs:  $KB$ , the knowledge base, a sentence in propositional logic  
            $\alpha$ , the query, a sentence in propositional logic  
  
   $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg\alpha$   
   $new \leftarrow \{ \}$   
  loop do  
    for each  $C_i, C_j$  in  $clauses$  do  
       $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )  
      if  $resolvents$  contains the empty clause then return true  
       $new \leftarrow new \cup resolvents$   
  if  $new \subseteq clauses$  then return false  
   $clauses \leftarrow clauses \cup new$ 
```

Can we do better?

- The resolution algorithm just requires a single inference rule, and it is **sound and complete** for propositional logic.
- However, notice we still have to resolve pairs over and over. This pair-resolution process is still finite, but can be exponential in worst-case time-complexity.
 - How many possible clauses are there if we have n symbols (literals)?
 - Each literal can be positive or negative (e.g. p and $\neg p$). So, total positive and negative literals is $2n$.
 - We have clauses of size 1, size 2, ..., size n .
 - So total is $\binom{2n}{1} + \binom{2n}{2} + \binom{2n}{3} + \dots + \binom{2n}{n} = O(2^n)$
- Can we do better?
- Yes, if the sentences in the KB are **Horn clauses**, then deciding entailment can be done in time that is **linear** in the size of the KB (linear in number of clauses and symbols in the KB).
- Two inference algorithms that operate on Horn clauses: **forward chaining** and **backward chaining**. They are **sound and complete**.

- A **Horn clause** is a disjunction of literals for which at *most* one is positive.
 - A *single* symbol that is a positive literal. Like P, Q, R
 - (conjunction of symbols) $\Rightarrow$ symbol. For example, $(P \wedge R) \Rightarrow Q$
- Horn clauses are nice because they can be written in the form of: *premises* $\Rightarrow$ *conclusion*
- Given a KB of just Horn clause, we can leverage the **Modus ponens** inference rule.

$$\frac{\alpha_1, \dots, \alpha_n, \quad \alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta}{\beta}$$

- Given a KB of horn clauses, we can do inference in linear time using forward chaining or backward chaining. Sound and complete algorithms.

Forward chaining algorithm via example

- To show $KB \models q$, forward chaining works as follows:
 - ❑ Apply Modus ponens: Begin from positive literals (facts) in the knowledge base. Find a definite clause with these positive literals in the premise, then add its conclusion to the KB.
 - ❑ Keep applying modus ponens until the query q is added to the KB or until no further inferences can be made.

- Consider the example to the right.
- You are given a KB with horn clauses.
- Note that a conjunction of horn clauses correspond to AND-OR graph.
- Use forward chaining to show that $KB \models Q$

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

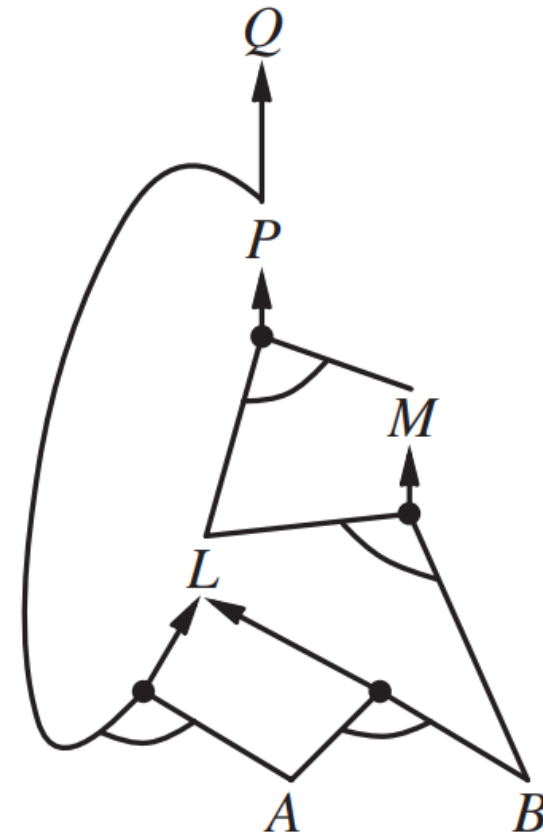
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining algorithm via example

- Use forward chaining to show that $KB \models Q$
- Start from positive literals. They become **active** in the AND-OR graph
- We will maintain a set of actives as we prove.

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

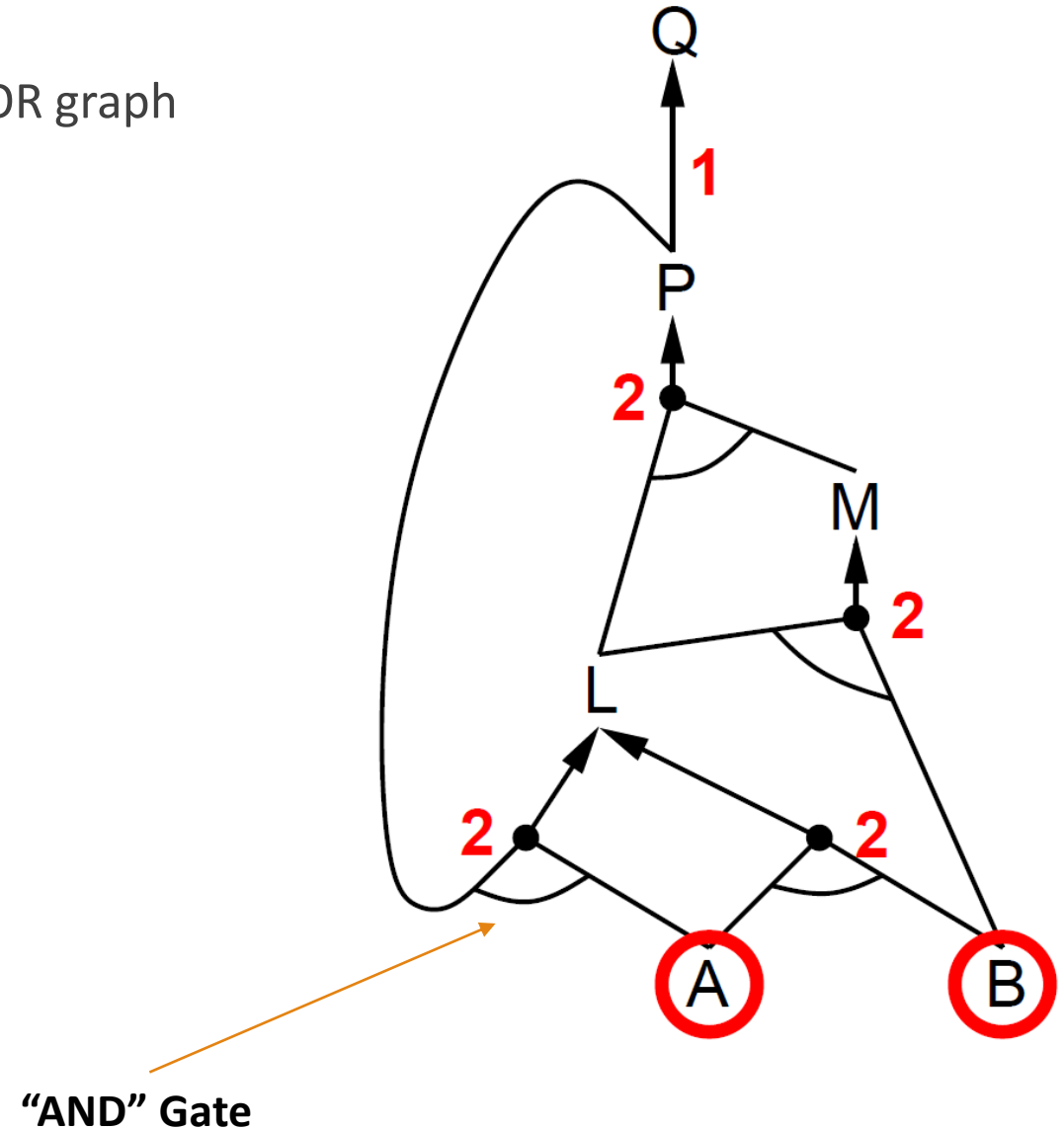
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining algorithm via example

- Use forward chaining to show that $KB \models Q$
- Activate A
- Actives = {A}

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

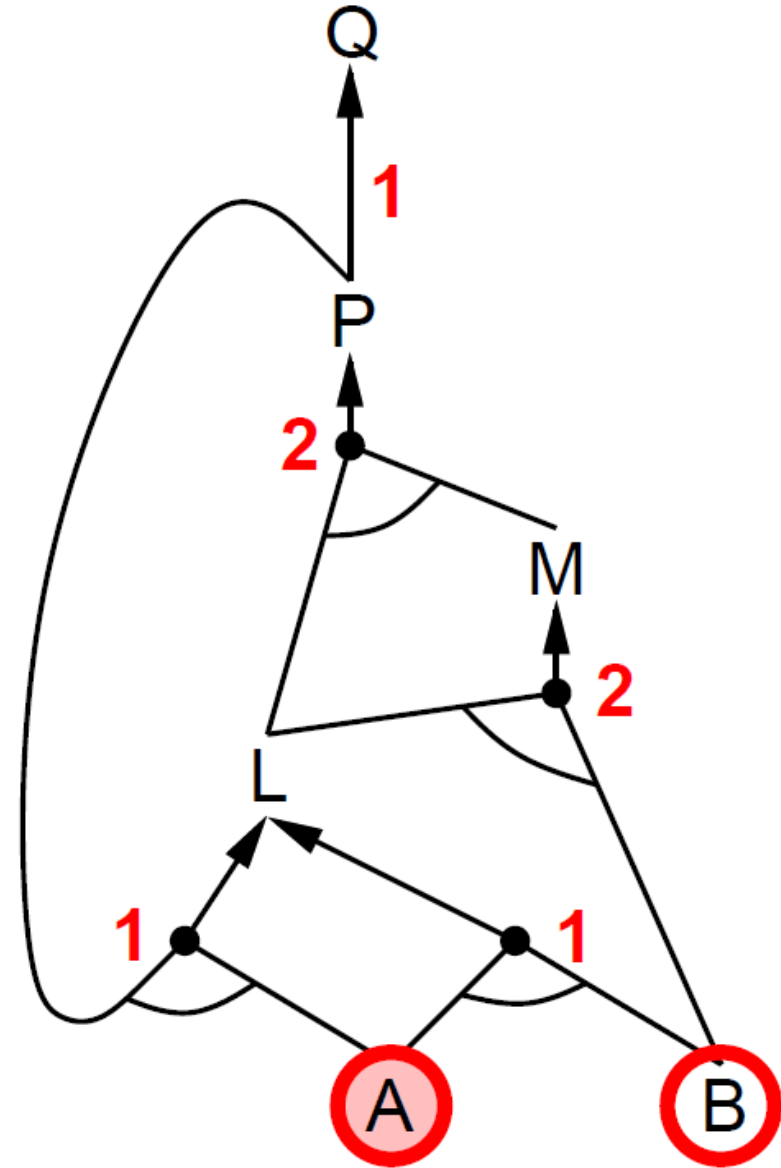
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining algorithm via example

- Use forward chaining to show that $KB \models Q$
- Activate B.
- Actives = {A, B}

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

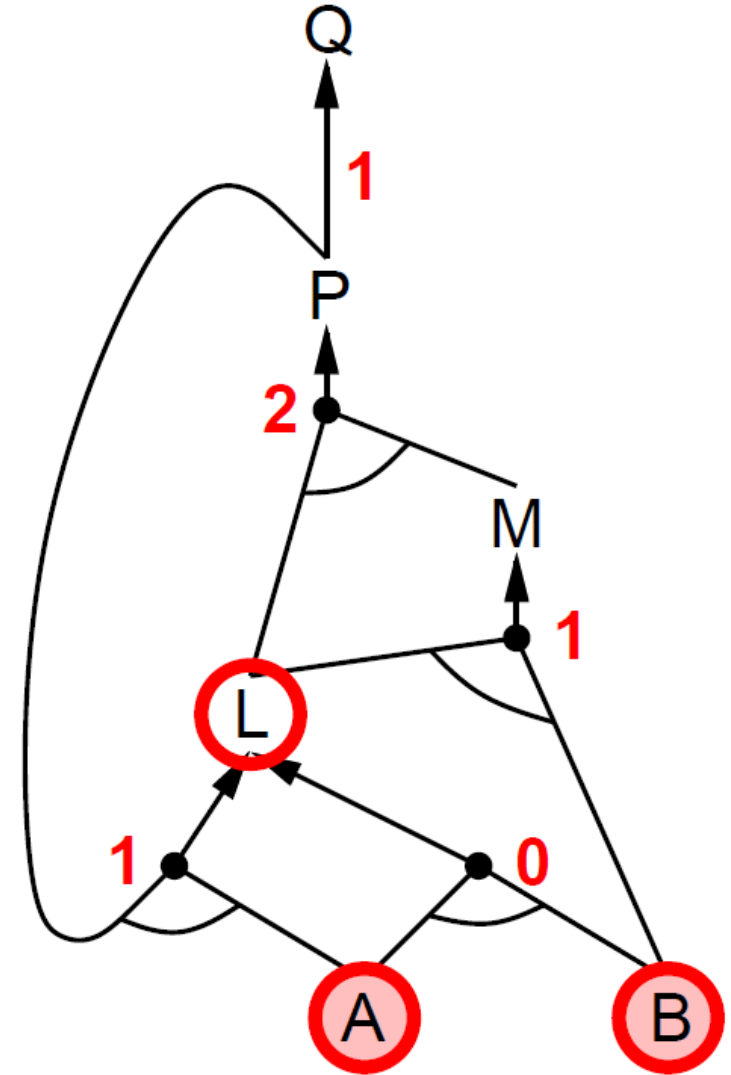
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining algorithm via example

- Use forward chaining to show that $KB \models Q$
- A and B in Actives, so L becomes active.
- Actives = {A, B, L}

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

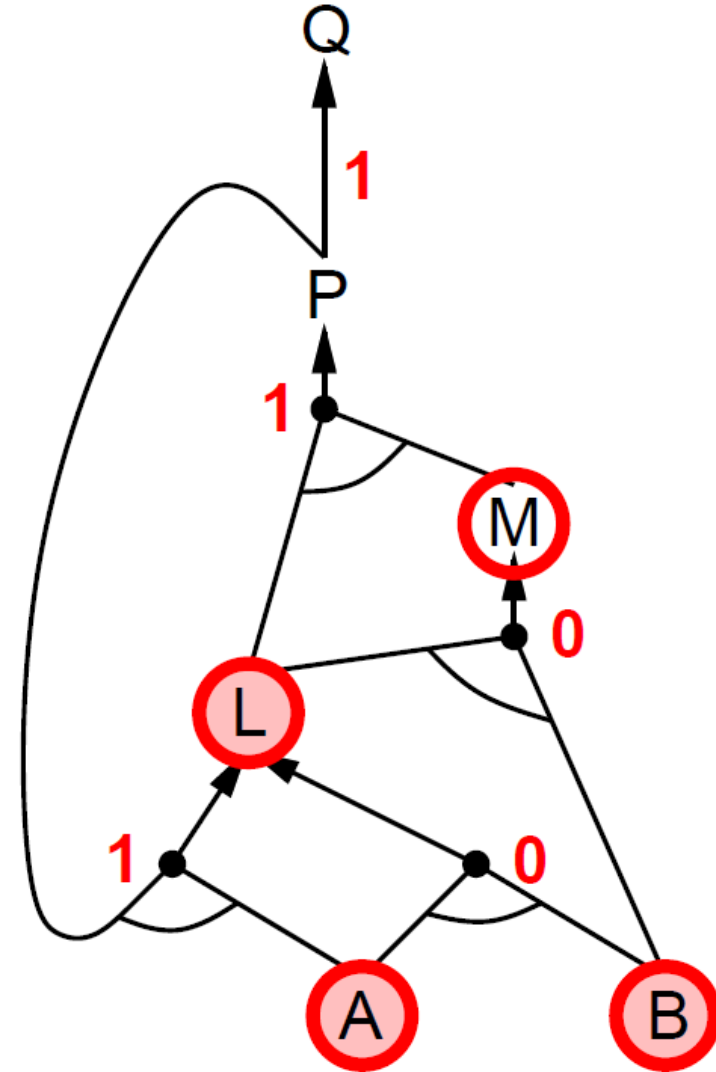
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining algorithm via example

- Use forward chaining to show that $KB \models Q$
- B and L in Actives, so M becomes active.
- Actives = {A, B, L, M}

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

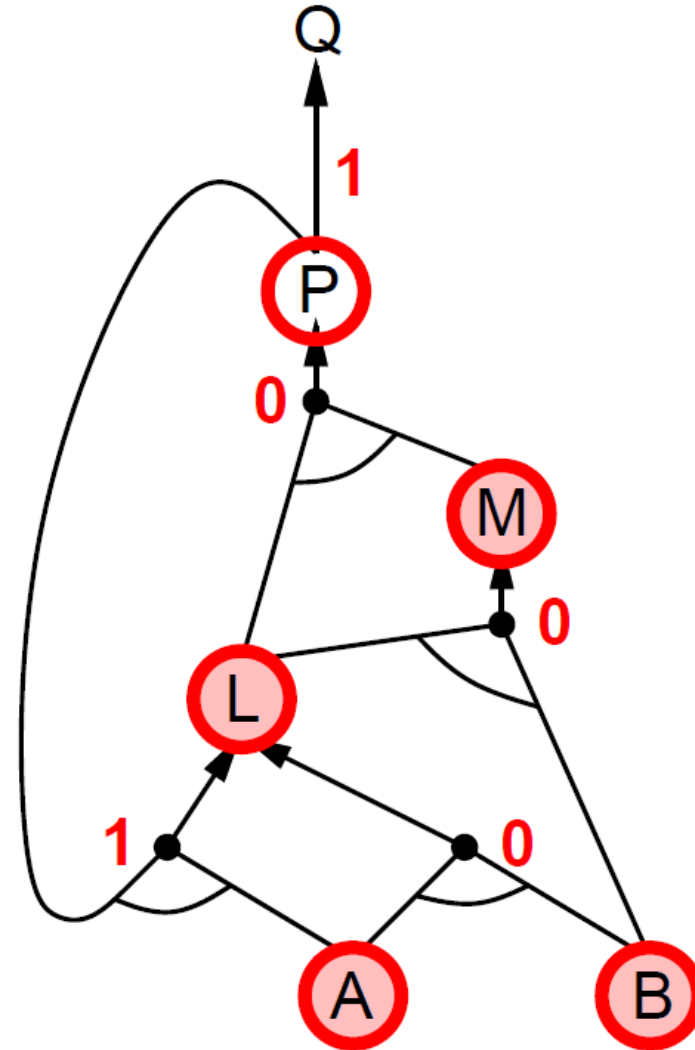
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining algorithm via example

- Use forward chaining to show that $KB \models Q$
- L and M in Actives, so P becomes active.
- Actives = {A, B, L, M, P}

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

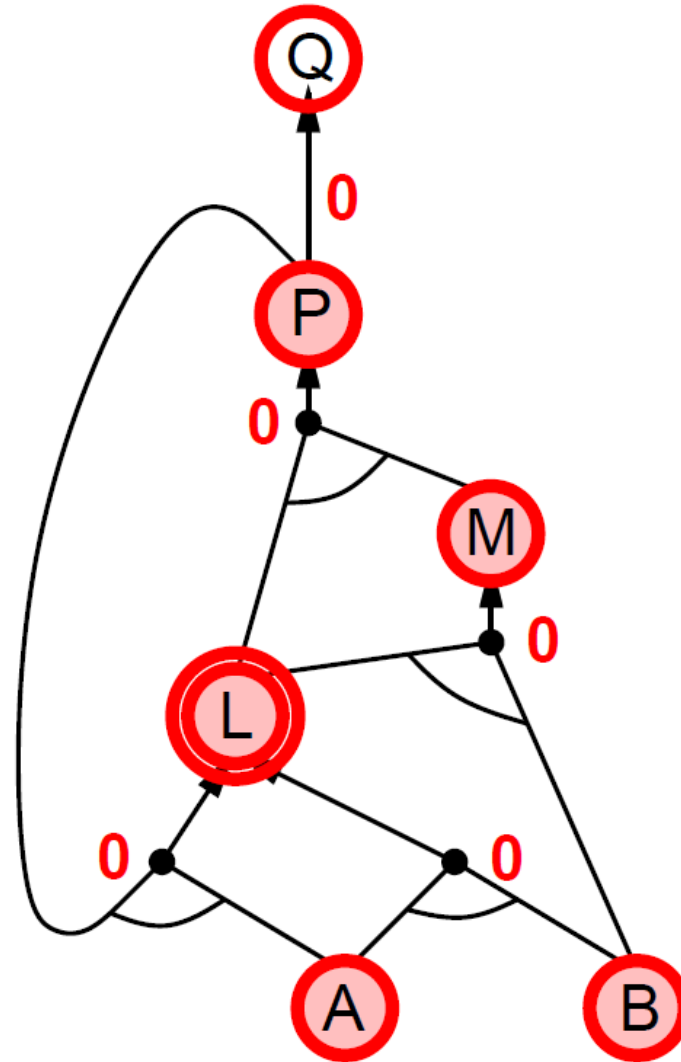
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining algorithm via example

- Use forward chaining to show that $KB \models Q$
- P in Actives, so Q becomes active.
- Actives = $\{A, B, L, M, P, Q\}$
- Since Q has become active, we are done. So, $KB \models Q$

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

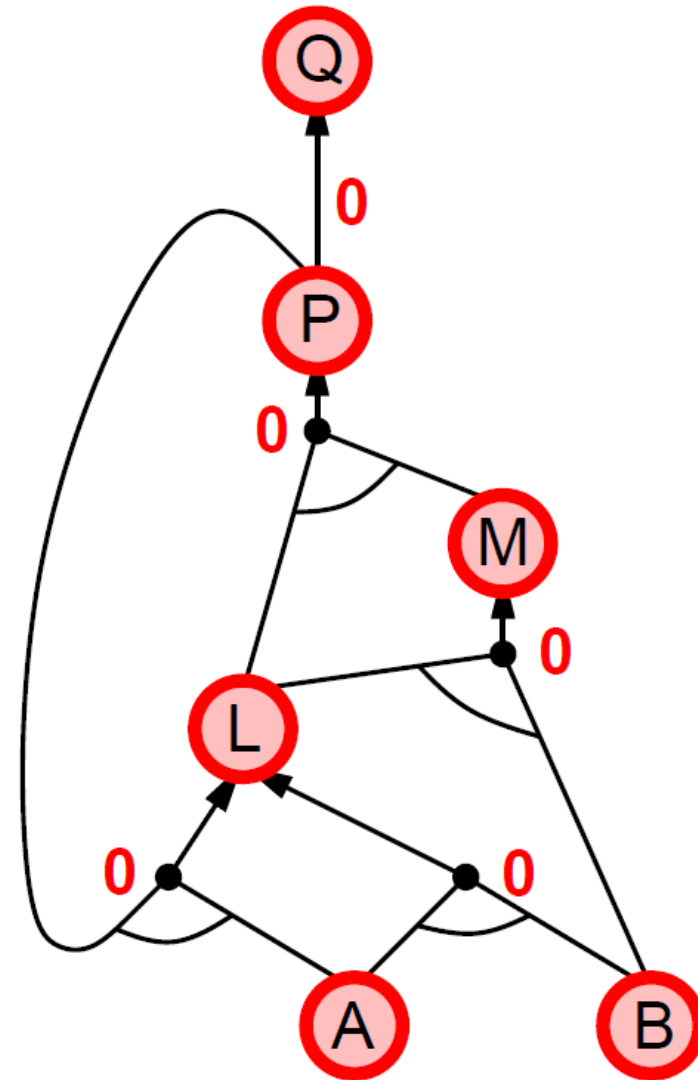
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- **Backward chaining** works backwards from the query q .
- Maintain a stack of sub-goals that need to be proved.
- Use backward chaining to show that $KB \models Q$.
- Already-proved = {A, B}. sub-goals = {Q}
- Q needs to be proved. So, start from $P \Rightarrow Q$.

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

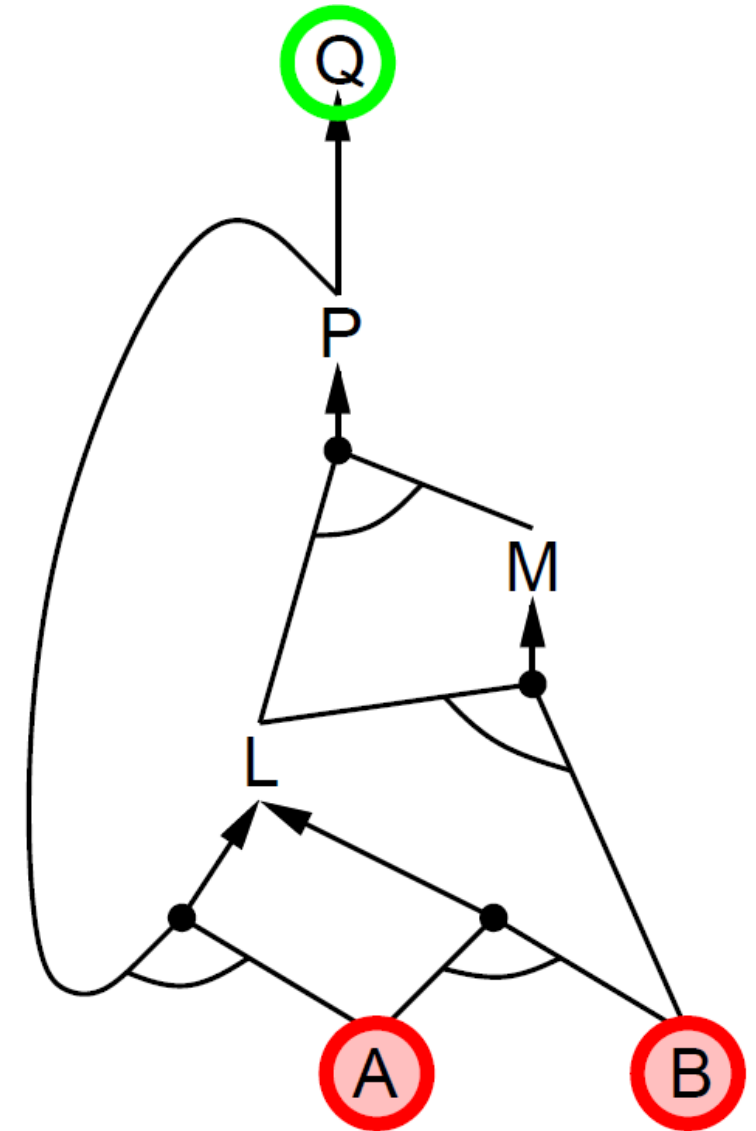
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- Inspect Q in $P \Rightarrow Q$. Need to prove P .
- Already-proved = $\{A, B\}$. sub-goals = $\{Q, P\}$

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

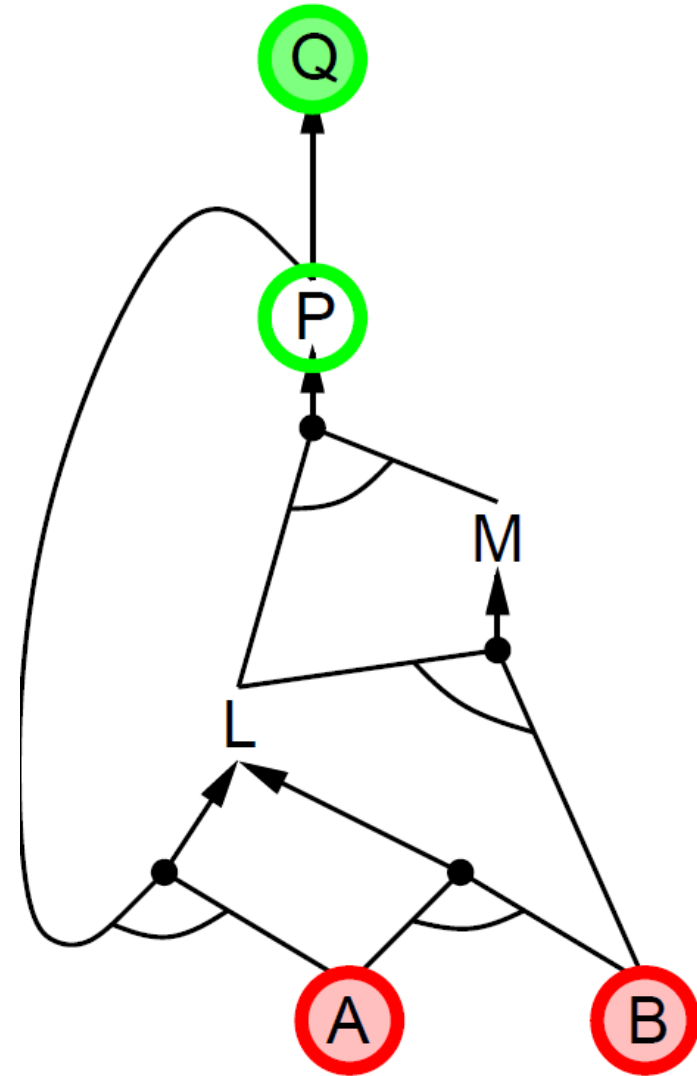
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- Inspect P in $L \wedge M \Rightarrow P$. Need to prove L and M.
- Already-proved = {A, B}. sub-goals = {Q, L, M, P}

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

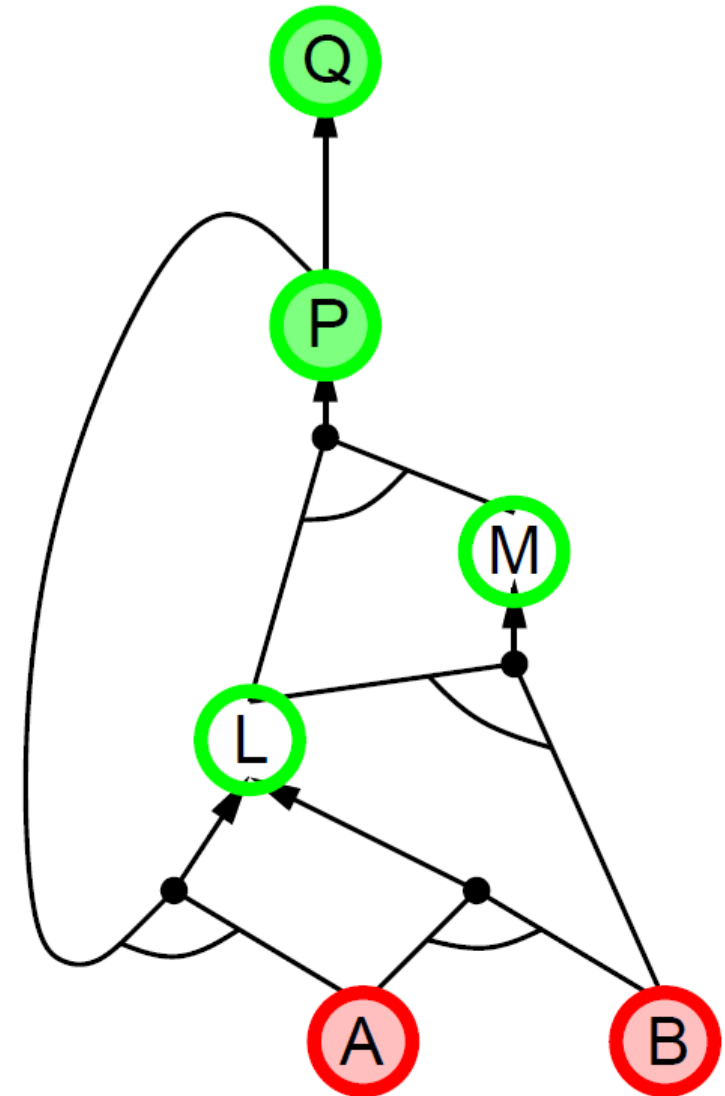
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- Inspect L in $A \wedge P \Rightarrow L$. A already proved, and P already in sub-goal.
- Already-proved = {A, B}. sub-goals = {Q, L, M, P}

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

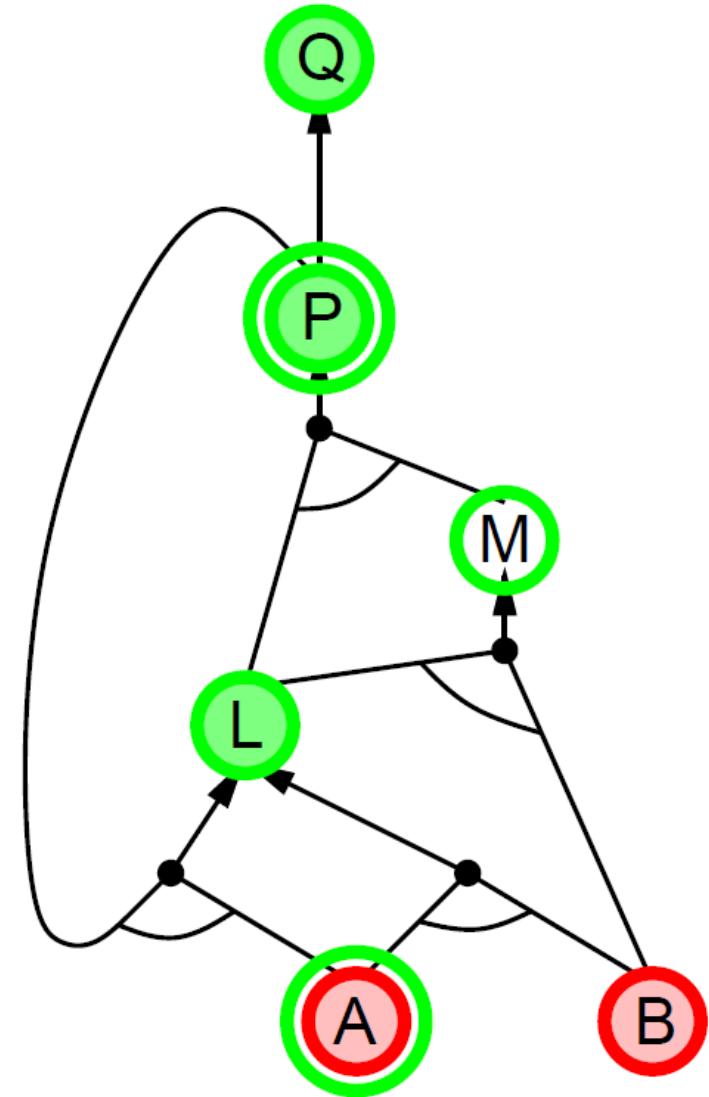
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- From $A \wedge B \Rightarrow L$. A and B proved, which proves L.
- Already-proved = {A, B, L}. sub-goals = {Q, P, M}.

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

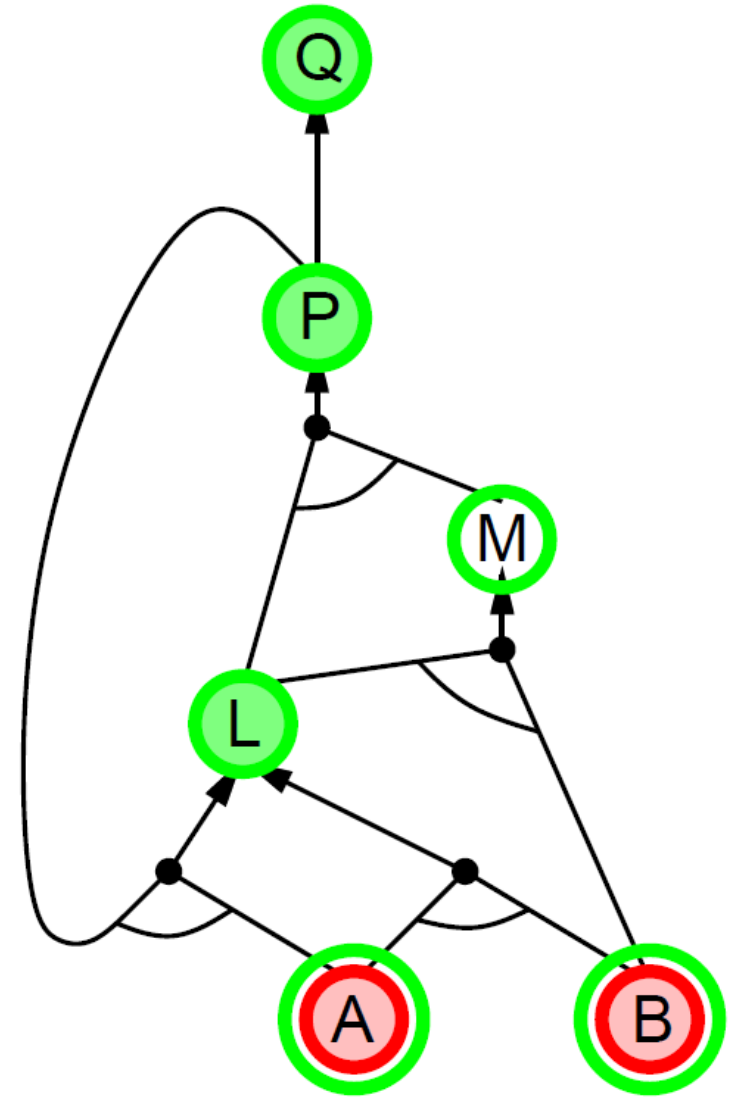
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- From $B \wedge L \Rightarrow M$. B and L proved, which proves M
- Already-proved = {A, B, L, M}. sub-goals = {Q, P}.

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

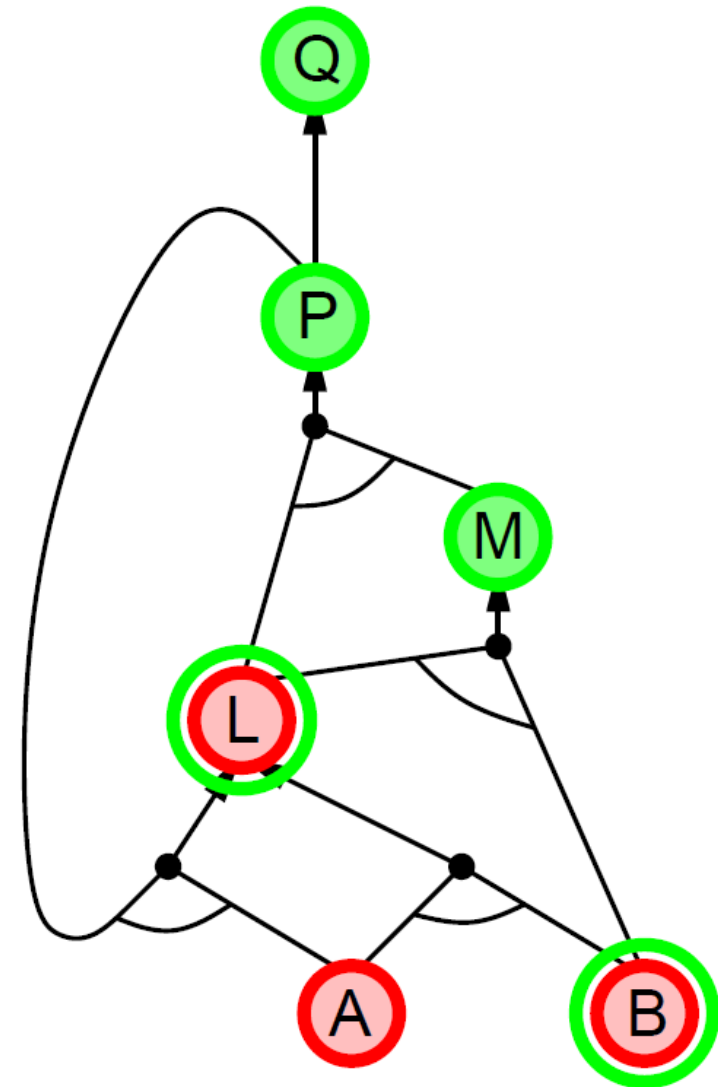
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- From $L \wedge M \Rightarrow P$. L and M proved, which proves P.
- Already-proved = {A, B, L, M, P}. sub-goals = {Q}.

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

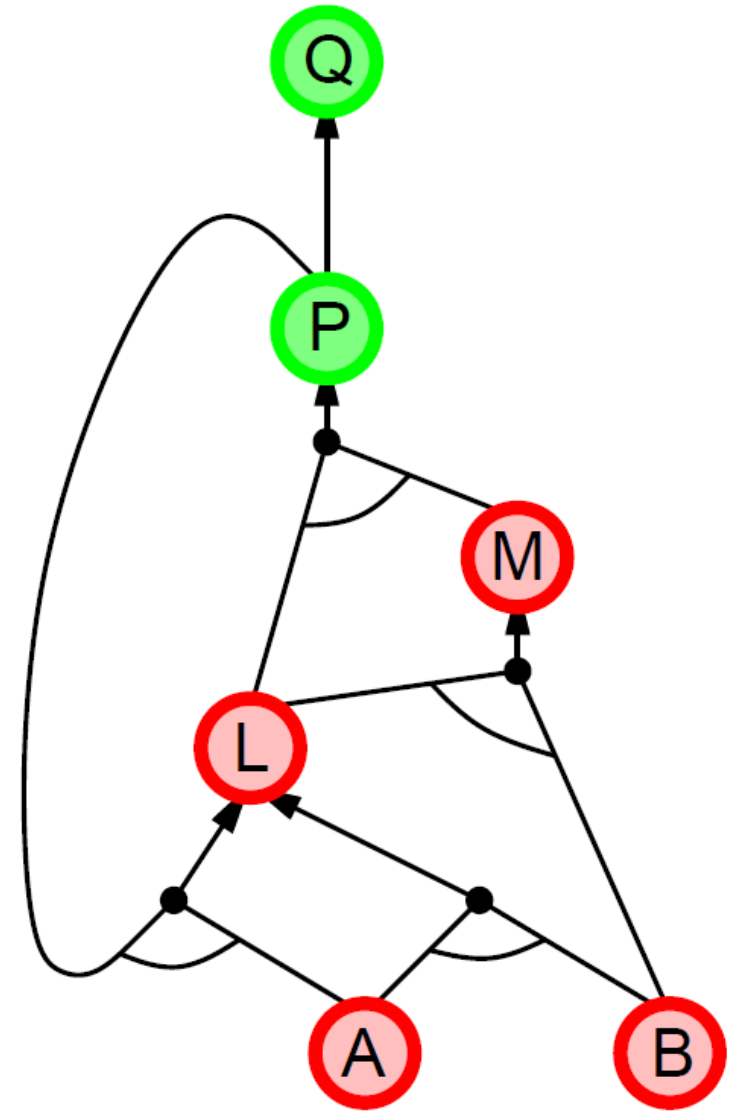
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Backward chaining algorithm via example

- Q is proved from P.
- Already-proved = {A, B, L, M, P, Q}. sub-goals = {}.
- Q is in proof list, so we are done. $KB \models Q$

KB sentences:

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

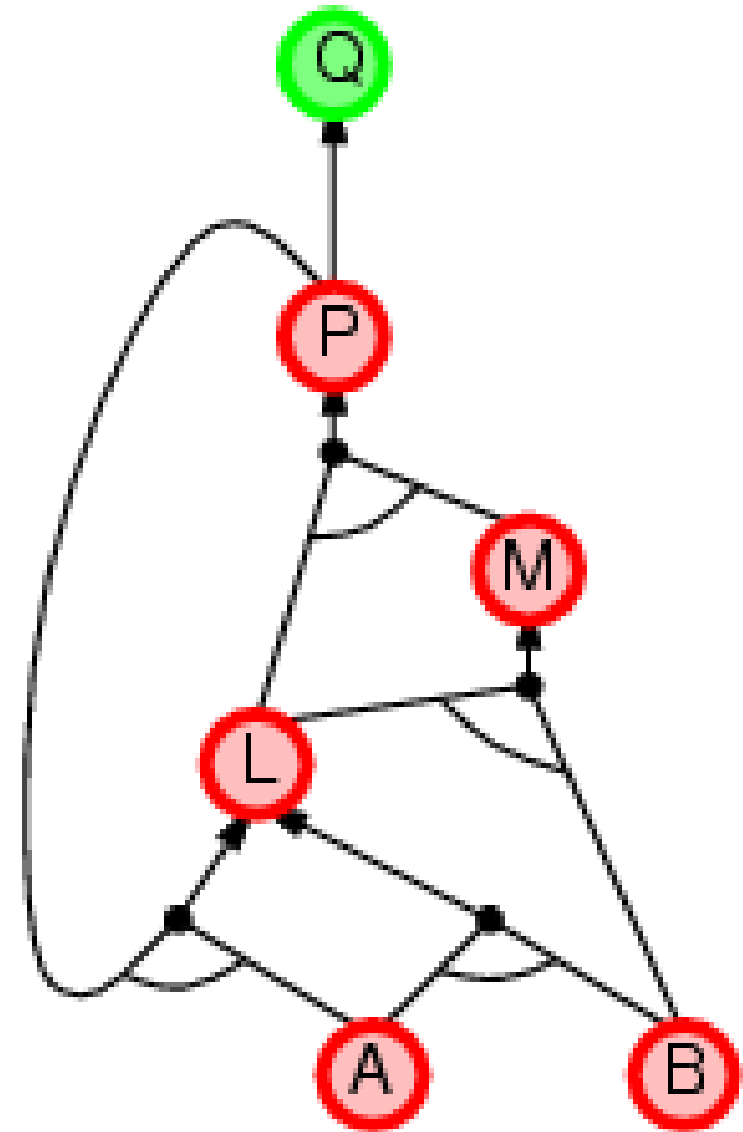
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward vs. backward chaining

- Both forward and backward chaining algorithms are sound and complete for Horn clauses.
- Both have worst-case time-complexity that is **linear** in size of KB (# clauses and # symbols).
- Backward chaining is **goal-driven**, it starts from the query and works backwards, proving only premises that are relevant for the query.
- Forward chaining is **data-driven**, it starts from known facts in the KB, then draws conclusions. These conclusions may be irrelevant to the query.
- In practice backward chaining can cost less than linear in size of KB since you might not need to process all clauses. Commonly use inference engine in logic programming languages like [Prolog](#), [Knowledge Machine](#) and [ECLiPSe](#).
- Many real-world problem can be naturally represented as horn clause KBs (premise => conclusion)

Forward Chaining Implementation

- Other implementations possible.
- Given a KB with $O(n)$ symbols and $O(k)$ clauses, the while loop is $O(n)$ and innerloop is $O(k)$, so total runtime is $O(nk)$.
- Backward chaining can be implemented in similar way.

function PL-FC-ENTAILS?(KB, q) **returns** *true* or *false*

inputs: KB , the knowledge base, a set of propositional definite clauses

q , the query, a proposition symbol

$count \leftarrow$ a table, where $count[c]$ is initially the number of symbols in clause c 's premise

$inferred \leftarrow$ a table, where $inferred[s]$ is initially *false* for all symbols

$queue \leftarrow$ a queue of symbols, initially symbols known to be true in KB

while $queue$ is not empty **do**

$p \leftarrow \text{POP}(queue)$

if $p = q$ **then return** *true*

if $inferred[p] = \text{false}$ **then**

$inferred[p] \leftarrow \text{true}$

for each clause c in KB where p is in c .PREMISE **do**

decrement $count[c]$

if $count[c] = 0$ **then** add c .CONCLUSION to $queue$

return *false*

Figure 7.15 The forward-chaining algorithm for propositional logic. The *agenda* keeps track of symbols known to be true but not yet “processed.” The *count* table keeps track of how many premises of each implication are not yet proven. Whenever a new symbol p from the agenda is processed, the count is reduced by one for each implication in whose premise p appears (easily identified in constant time with appropriate indexing.) If a count reaches zero, all the premises of the implication are known, so its conclusion can be added to the agenda. Finally, we need to keep track of which symbols have been processed; a symbol that is already in the set of inferred symbols need not be added to the agenda again. This avoids redundant work and prevents loops caused by implications such as $P \Rightarrow Q$ and $Q \Rightarrow P$.

Summary of inference algorithms in Propositional Logic

- We have shown the following inference algorithms to establish entailment in propositional logic:
 - ❑ Model checking: truth-table enumeration. Costs $O(2^n)$ for n propositional symbols. Sound and complete.
 - ❑ Direct proof using a search algorithm with all inference rules. **Costs as much as the search algorithm used (still exponential time complexity)**. Soundness and completeness depends on the inference rules used and the search algorithm used, respectively. If the inference rules used are enough for deriving the query, then this method is sound. If the search algorithm used has completeness, then this method is complete. Note that branching factor is max number of applicable inference rules at any given state.
 - ❑ Proof by contradiction using Resolution-based algorithm with CNF KB. Sound and complete. Worst-case exponential; *practically* can be much less than model checking.
 - ❑ Forward/backward chaining with horn clauses KB. Sound and complete. **Linear** in size of KB. Most KB's can be built naturally with horn clauses.
- Are there more inference algorithms? Yes (Modern SAT Solvers). There are backtracking DFS-type algorithms (DPLL; worst-case still exponential; can be fast in practice), and local search approaches. See section 7.6 in textbook.
- For a general KB: *“Unfortunately, propositional entailment is co-NP-Complete (probably no easier than NP-Complete). Every known inference algorithm for entailment in propositional logic has a **worst-case** complexity that is exponential in size of input.”*

SAT Solvers

Boolean Satisfiability (SAT)

- Given a formula, find an assignment of True/False to the variables that satisfies the entire formula (makes the formula evaluate to True)

$$x_1 \wedge \neg x_2 \vee \neg x_4 \wedge x_3 \wedge \dots \vee x_n$$

- SAT is an NP-Complete problem. No known efficient algorithm for all cases.
- In practice, many formulas can be solved quickly using modern solvers.
- Why is SAT relevant? Many problems can be modelled as SAT. Satisfying the modelled formula gives solution to the original problem.
- For propositional entailment, we can use a SAT solver to determine if $(KB \wedge \neg \alpha)$ is unsatisfiable.
- Rough summary of SAT solvers:
 - Simple SAT solver that does a DFS tree enumeration of all possible models. (we've seen this algorithm in model-checking)
 - Davis–Putnam–Logemann–Loveland (DPLL). Proposed in 1962. Still a DFS tree enumeration, but uses clever tricks to reduce amount of the subtrees checked in the full tree. Similar to backtracking from CSPs. Not guaranteed to reduce; worst-case still $O(2^n)$. We'll briefly explain how this algorithm works.
 - Modern SAT Solvers. Initially most are inspired by DPLL (1962), and more recent ones inspired by [CDCL](#) technique (1996). Popular research domain. Adds various tricks, caching, and heuristics to reduce amount of work. Modern solvers can handle several millions of variables. Nice overview [talk](#) for those interested.
 - Note: SAT Solvers assume input is a CNF formula. (convert to CNF, then run SAT Solver)

Simple SAT Solver

$$f(a, b, c, d) = (a \vee b \vee c) \cdot (a \vee b \vee \bar{c}) \cdot (\bar{a} \vee c \vee d) \cdot (\bar{a} \vee c \vee \bar{d}) \cdot (\bar{b} \vee \bar{c} \vee d) \cdot (\bar{b} \vee \bar{c} \vee \bar{d})$$

- Simple SAT solver that does a DFS tree enumeration of all possible models. (we've seen this algorithm in model-checking)
- Time complexity $O(2^n)$
- Space complexity $O(n)$

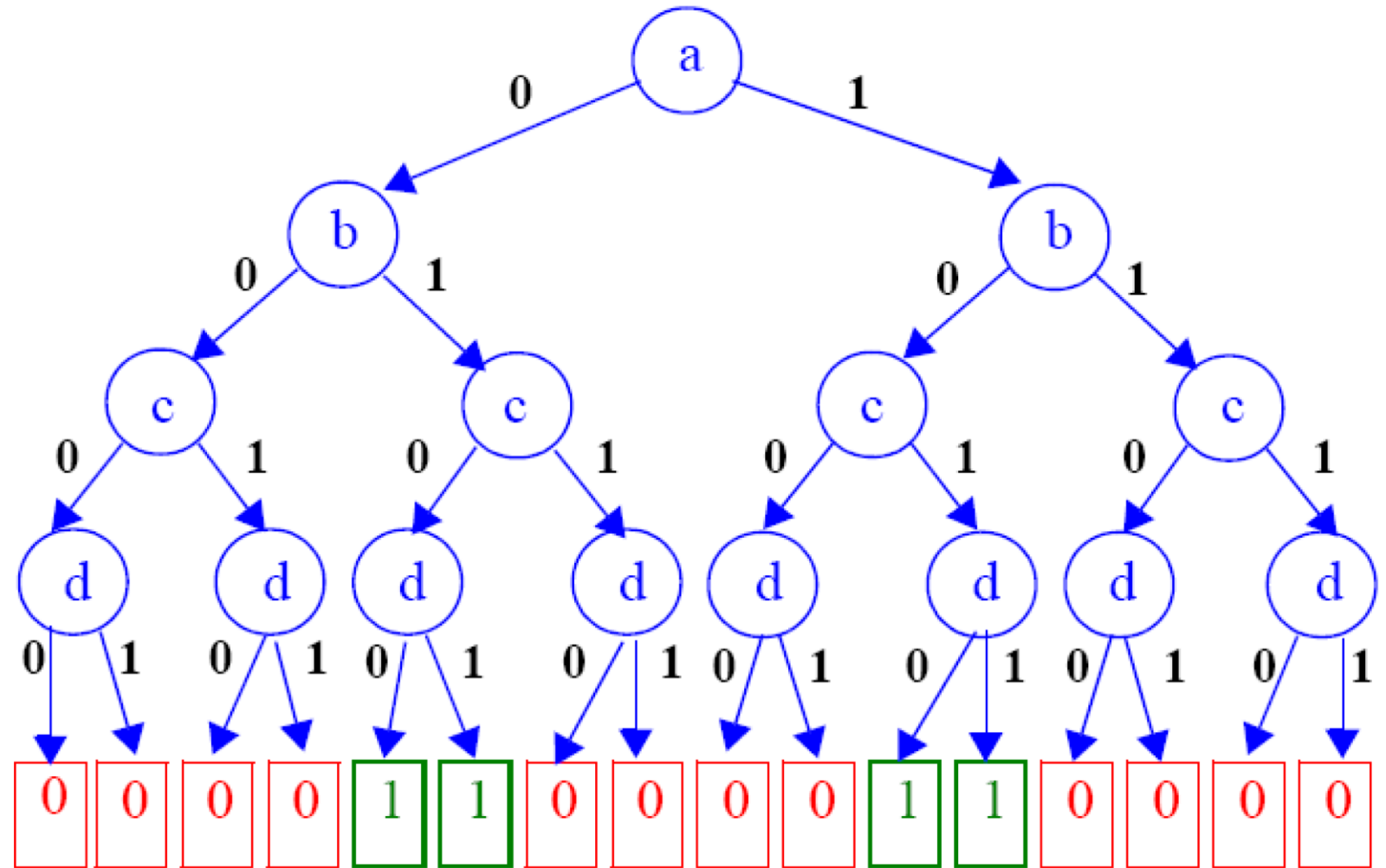


Image from [link](#)

- DPLL algorithm proceeds in DFS manner. At **each** node, it uses the following tricks/techniques:

- **Simplification:** When a variable is assigned true/false value, the clauses in the formula are simplified.

For example, if we assign $b = \text{True}$ to the formula: $(a \vee b \vee \neg c) \wedge (a \vee \neg b) \wedge (c \vee \neg b)$, it simplifies to $(a) \wedge (c)$.

- **Unit propagation:** If a clause is a unit clause (contains a single literal), this clause can only be satisfied by assigning the necessary value to make this literal true. Thus, no choice is necessary. Unit propagation also consists of *simplifying* the clauses after literal assignment. (removing every clause containing the literal and discarding the complement from every clause containing the complement).

For example, suppose we are down a path in the tree, and we have the following formula:

$(a \vee b \vee \neg c) \wedge (a \vee \neg b) \wedge (\neg b)$, we have the unit clause $(\neg b)$. To get satisfiability, we *must* assign $b = \text{False}$. This then propagates and simplifies the formula to: $(a \vee \neg c)$. In practice, we can get more unit clauses after applying unit propagation, so we can repeat this unit propagation again and again, speeding up the arrival to satisfiability or unsatisfiability.

- **Pure Literal Assignment:** If a propositional variable occurs with only one *polarity* in the formula, it is called pure. A pure literal can always be assigned in a way that makes all clauses containing it true. After assignment, we can simplify the formula (removing the clauses containing the pure literal).

a is a pure literal in the formula $(a \vee b \vee \neg c) \wedge (a \vee \neg b) \wedge (\neg b \vee c)$. Assign pure literals based on their sign, so here set $a = \text{True}$ and simplify the formula to: $(\neg b \vee c)$. Note now we get $\neg b$ as a pure literal, and also c .

DPLL [2]

- Recursive pseudocode for DPLL.
- Extra work is done within each node to reduce the overall search work.
- Similar to CSP backtracking with heuristics and filtering, these extra steps *may* help in reducing the search work. Worst-case is still $O(2^n)$.
- In practice, for many formulas gives huge speedup.

```
def DPLL(clauses)
    # unit propagation, includes simplifying clauses
    while there is a unit clause c in clauses:
        clauses = unit_propagation(c, clauses)

    # pure literal elimination, includes simplifying clauses
    while there is a literal L that occurs pure in clauses:
        clauses = pure_literal_assignment(L, clauses)

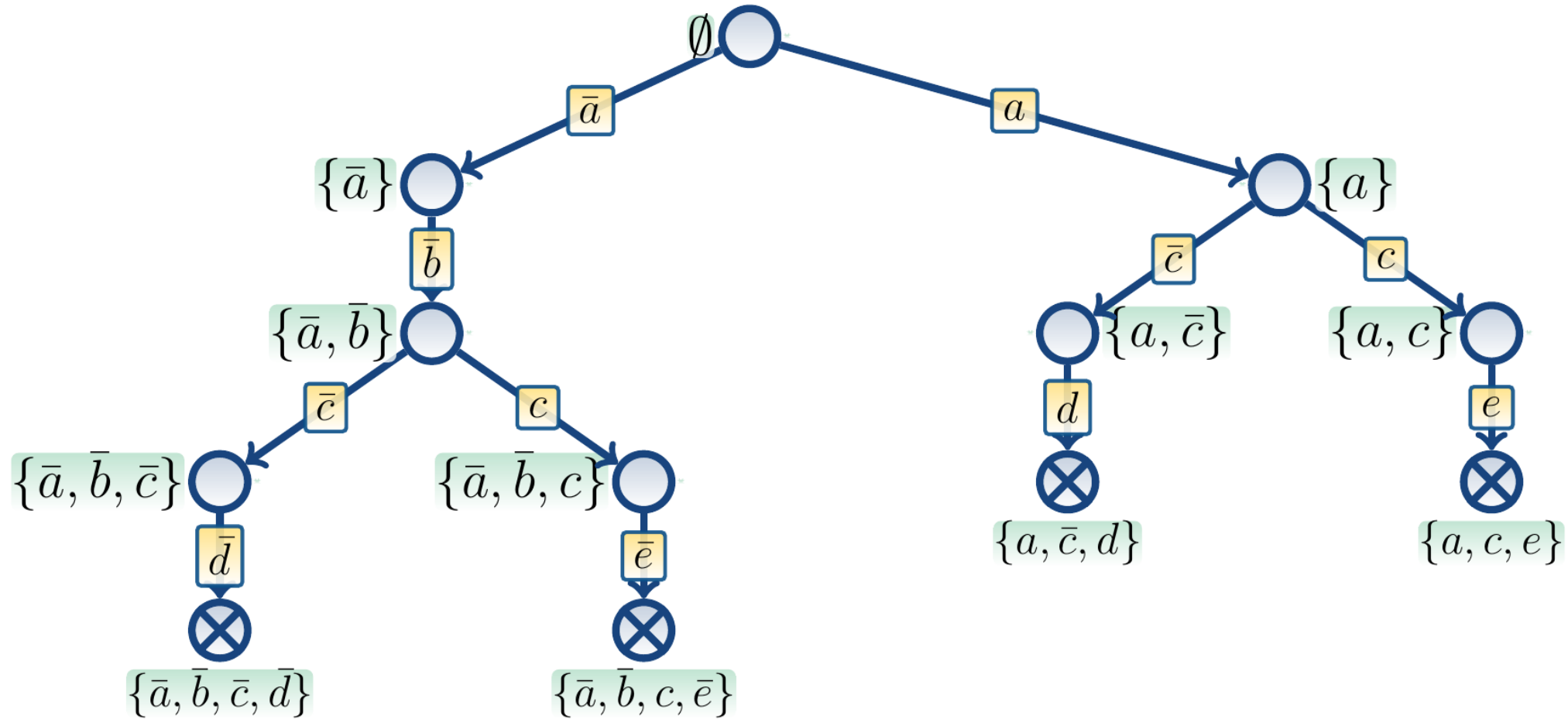
    # stop criteria
    # after simplifications, empty clauses means all clauses are satisfied
    if clauses is empty:
        return True

    # after simplifications, a false clause means unsatisfied formula
    if clauses contains False-clause:
        return False

    # otherwise stop condition not met yet, so proceed down the tree
    L = choose_literal_to_assign(clauses)
    return DPLL( simplify_clauses(clauses, L) ) or DPLL( simplify_clauses(clauses, not(L)) )
```

DPLL Example

$$(a \vee \neg b) \wedge (\neg a \vee c \vee \neg d) \wedge (a \vee c \vee \neg d) \wedge (\neg c \vee \neg e) \wedge (\neg c \vee e) \wedge (c \vee d)$$



- Unit-propagation: After branching on $\neg a$, unit propagation of the clause $(a \vee \neg b)$ forces $\neg b$.
- Simplification when we branched on a , notice that we no longer need to consider b branches.

Example
from: [link](#)

PySAT Package

- [PySAT](#) is python package that contains various solvers. Supports PL formulas. Can extend to implement your own solvers.

```
from pysat.formula import CNF
from pysat.solvers import Solver

# create a satisfiable CNF formula "(-x1 V x2) ∧ (-x1 V -x2)":
cnf = CNF(from_clauses=[[-1, 2], [-1, -2]])

# create a SAT solver for this formula:
with Solver(bootstrap_with=cnf) as solver:
    # 1.1 call the solver for this formula:
    is_satisfying = solver.solve()
    sat_model = solver.get_model()

    print('formula is', f'{"s" if is_satisfying else "uns"}atisfiable')
    print('and the model is:', solver.get_model())

formula is satisfiable
and the model is: [-1, -2]
```

PySAT Package

- Supports a good number of modern solvers.

`class pysat.solvers.SolverNames`

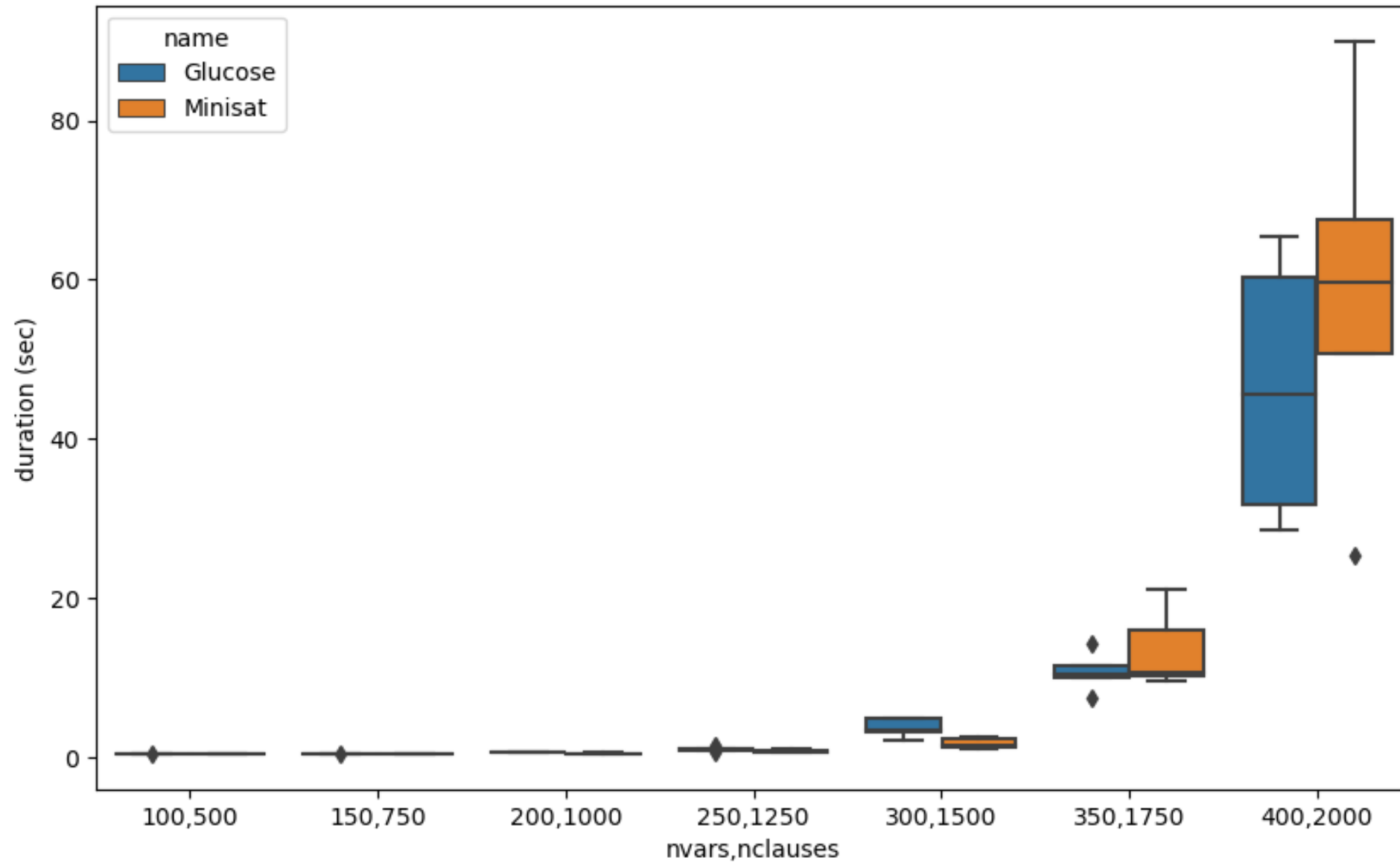
This class serves to determine the solver requested by a user given a string name. This allows for using several possible names for specifying a solver.

```
cadical103 = ('cd', 'cd103', 'cd1', 'cdl103', 'cadical103')
cadical153 = ('cd15', 'cd153', 'cdl15', 'cdl153', 'cadical153')
cryptosat = ('cms', 'cms5', 'crypto', 'crypto5', 'cryptominisat', 'cryptominisat5')
gluecard3  = ('gc3', 'gc30', 'gluecard3', 'gluecard30')
gluecard41 = ('gc4', 'gc41', 'gluecard4', 'gluecard41')
glucose3   = ('g3', 'g30', 'glucose3', 'glucose30')
glucose4   = ('g4', 'g41', 'glucose4', 'glucose41')
glucose42  = ('g42', 'g421', 'glucose42', 'glucose421')
lingeling  = ('lgl', 'lingeling')
maplechno  = ('mcb', 'chrono', 'maplechno')
maplecm    = ('mcm', 'maplecm')
maplesat   = ('mpl', 'maple', 'maplesat')
mergesat3  = ('mg3', 'mgs3', 'mergesat3', 'mergesat30')
minicard   = ('mc', 'mcard', 'minicard')
minisat22  = ('m22', 'msat22', 'minisat22')
minisatgh  = ('mgh', 'msat-gh', 'minisat-gh')
```

Runtime Comparisons [1]

- [PySAT](#) is python package that contains various solvers. Supports PL formulas. Can extend to implement your own solvers.
- Experimental setup:
 - Generate unsatisfiable random 3-CNFs.
 - Use two modern solvers (Glucose and Minisat); limit to 60 seconds.
 - Compare runtime execution.
 - Note this is not a thorough comparison.
 - Did not include simple SAT solver and DPLL due to time limit. They start to struggle after 20 variables.

Runtime Comparisons [2]



After PL

Pros and Cons of PL

- Pros of PL:
 - Declarative: symbols correspond to facts.
 - Allows partial, disjunctive, and negated information. E.g. “there is pit in [1,2] or [2,1]”.
 - Compositional: meaning of $A \wedge B$ is derived from meaning of A and meaning of B.
 - Inference procedure is **domain-independent**.
 - Inference procedures exist that are sound and complete.
- Main drawback of PL is its **limited expressive power**.
 - Recall Wumpus world example. To say “pits cause breezes in adjacent squares” required a sentence for each square.
$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$
 - Can't deal with change in facts over time (can use symbol per time step but this compounds # of symbols)
 - Difficult to represent objects, relationships, and properties.
- Need a more expressive language with inference capabilities: First-order logic (FOL)
- Need a framework to account for uncertainty: Probability Theory

Formal Logic Languages

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

- We discussed propositional logic (Ch. 7).
- FOL: Ch. 8 and 9 builds upon Ch. 7 and talks about a more flexible/expressive logic.
- Probability Theory: Ch. 12 and 13 builds upon Ch. 7 and considers facts with uncertainty.

Wumpus World KB in FOL

- Expressing these rules in FOL is much more concise.
- A typical percept sentence would be:

`Percept([Stench, Breeze, Glitter , None, None], 5)`

Percept here is a binary predicate. Constants are Stench, Breeze, Glitter, etc. The number indicates timestep.

- Actions can be represented by:

`Turn(Right), Turn(Left), Forward, Shoot, Grab, Climb`

- Perception rules: percepts implies facts:

$\forall t, s, g, m, c \text{ Percept}([s, \text{Breeze}, g, m, c], t) \Rightarrow \text{Breeze}(t)$

$\forall t, s, b, m, c \text{ Percept}([s, b, \text{Glitter}, m, c], t) \Rightarrow \text{Glitter}(t)$

- Simple reflex behaviour at timestep t:

$\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$

- With this behaviour, suppose the agent perceives: `Percept([Stench, Breeze, Glitter , None, None], 5)` which is added to the KB. We can then ask “ $\text{KB} \models \exists a \text{ BestAction}(a, 5)$?”
- Using perception rule and reflex behavior, the KB would return the substitution `{a/Grab}`

Wumpus World KB in FOL

- Agent's location s at time t can be indicated with: $At(\text{Agent}, s, t)$
- Adjacency of any two squares can be defined as:
$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$
- Unary predicate to indicate that a location s is a pit: $Pit(s)$
- Given its current location, the agent can infer properties of the square from properties of its current percept
$$\forall s, t \text{ At}(\text{Agent}, s, t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$$
- Having discovered which places are breezy (or smelly) and, very important, not breezy (or not smelly), the agent can deduce where the pits are (and where the Wumpus is).
- With PL, we would need propositions for each timestep.
 - For $N \times N$ grid, we need N^2 propositions for pits, $4N^2$ propositions for breeze mechanics, N^2 propositions for wumpus, $4N^2$ propositions for stinky mechanics, N^2 propositions for gold glitter.
 - For max of T timesteps
 - That is total of around $12TN^2$. With $N = 100, T = 100$, that is over 1 Million variables.

Summary of Entailment in FOL

- In propositional logic, we saw the resolution algorithm is sound and complete.
 - If $KB \models \alpha$, then algorithm generates empty clause.
 - If $KB \not\models \alpha$, then algorithm will terminate because no more new clauses to generate.
- However, in FOL, resolution is **refutation-complete**. This means that for any formula in FOL:
 - If $KB \models \alpha$, then a resolution-based algorithm will generate the empty clause.
 - **For some formulas, if $KB \not\models \alpha$, then the algorithm may never terminate.**
- For those interested, this has to do with the decidability of the logic language.
 - Propositional logic is decidable.
 - FOL is semi-decidable. “Algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every nonentailed sentence” – From R&N Book Pg 300
- Prolog the logic programming language supports FOL:
 - Uses Horn clause representation.
 - Uses backward chaining.
 - Due to the way backward chaining is implemented in Prolog, can get stuck in infinite loops for some entailed queries.

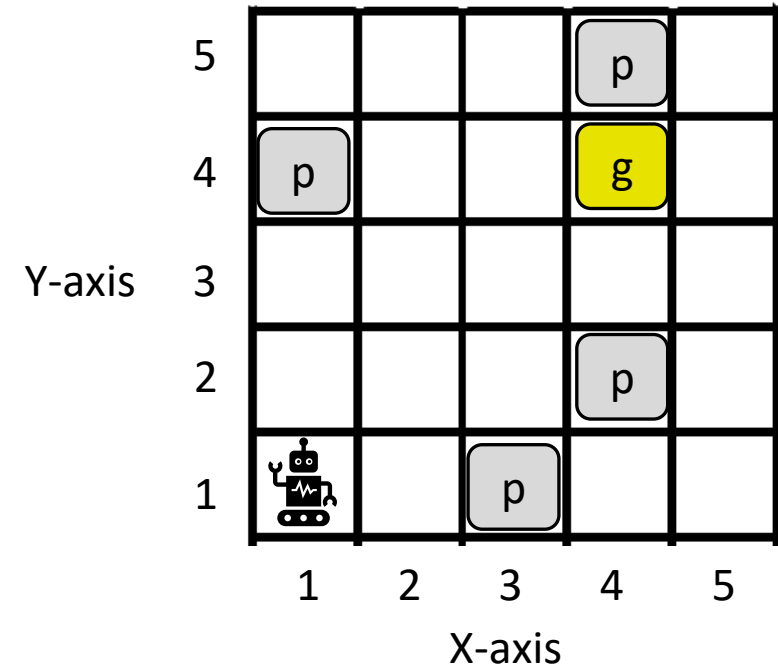
Using Prolog as Inference Engine

Prolog

- [Prolog](#) is a logic programming language. Supports a variant of first order logic (and other paradigms as well).
- Prolog [download page](#). Prolog tutorial [website](#).
- Prolog fundamentals (in no particular order):
 - Predicates and Objects
 - Constants vs variables, and variable unification.
 - Facts and rules to construct a KB.
 - Operators $\wedge$, $\vee$, and $\Rightarrow$, negation. Special true and false.
 - Querying
- Note1: Prolog querying used to return yes/no (entailment), but later implementations return true/false. In our case, when we query we treat true as yes, and false as no.
- Note2: Prolog is based on closed world assumption (CWA) -- that is, if a proposition is not in the knowledge base or not derivable from the knowledge base, then it is assumed to be not true (false). This can be problematic when using entailment.
- Let's do some practice.

Pit World using Prolog

- Consider a 5x5 grid environment in figure:
 - Agent starts in [1,1] and knows this.
 - Agent can move up, down, left, right.
 - There is gold one grid location that emits glitter.
 - There are pits that emit breeze in adjacent grids.
 - Agent does not know location of gold and pits.
 - Agent can sense in its location [breeze, glitter].
 - Goal: get gold safely and get out.
- We want to use Prolog to:
 - Model these facts and rules to construct an initial KB.
 - Query the KB which grid locations are safe so that we can explore them.
 - Perform actions in the environment (not using the KB). Gather sensor data, then update the KB with new facts and erase/update old facts.
 - Continue loop of query -> perform action -> update KB -> ... until goal.
- Note: For this given grid setup, it is possible to find safe path using logic (true/false) reasoning. However, for other grid setups, logic is not enough; using probability we can pick the least risk action.



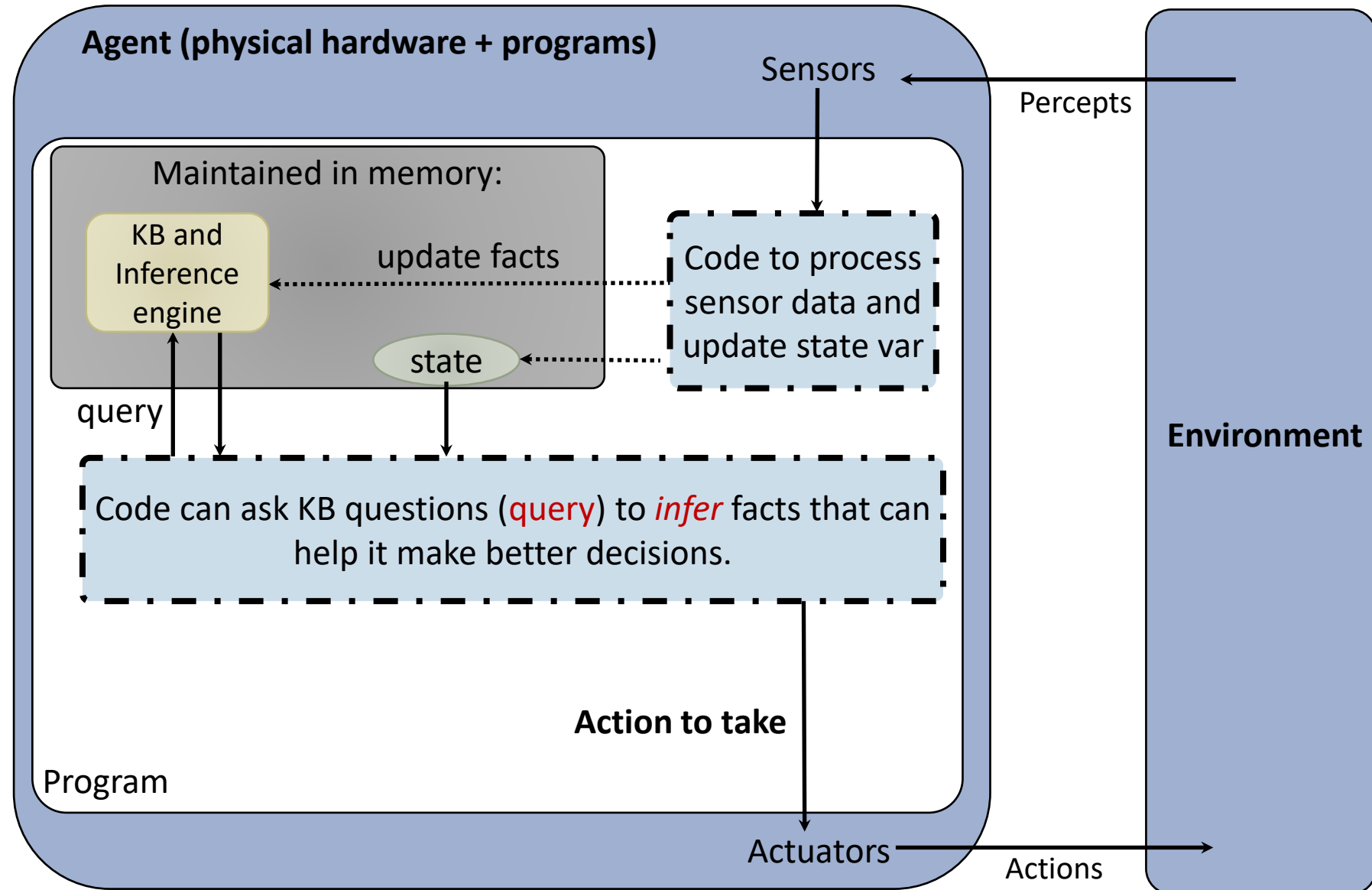
Rows

5				p	
4	p			g	
3					
2				p	
1			p		
	1	2	3	4	5

columns

KB- Agent

- Pit world example showed a basic setup of agent code interacting with a KB and inference engine like Prolog to make progress in an environment.
- We are not limited to using Prolog for our inference engine. Can use whatever is suitable and efficient. Other options exist like using SAT solver.



Summary

Summary

- Logical agents apply **inference** to a **knowledge base** to derive new information and make decisions.
- Basic concepts of logic:
 - **syntax** : formal structure of **sentences**
 - **semantics** : truth of sentences w.r.t. **models**
 - **entailment** : necessary truth of one sentence given another
 - **inference** : deriving sentences from other sentences
 - **soundness** : derivations produce only entailed sentences
 - **completeness** : derivations can produce all entailed sentences
 - **valid**: sentence is true in every model (a tautology)
- Wumpus world requires ability to represent partial and negated information, reason by cases, etc.
- **Resolution** is complete for propositional logic: FC, BC are linear time, complete for Horn clauses.
- Propositional logic **lacks** expressive power and cannot account for uncertainty.